

**Software Architecture**

# **Flexibility and Microservices**

12

## **Contents**

- API Evolution: To break or not to break
- Layers of defense against change
- Extensibility and Plugins
- Microservices
- Feature Toggles
- Splitting the Monolith

This lecture is organized following some of the quality attributes a software architecture can have. We started from many design-time qualities. We've seen deployability as we made the transition to operations and runtime. Then we talked about scalability and availability. Today we close the cycle as we look at how easy it is to change a software architecture. This goes under the overall umbrella term of flexibility.

We will see that the construct that is meant to make it easy to change a software architecture is called: "microservice".

This is the latest addition to our zoo of architectural concepts, starting from the good old days of component based software engineering, followed by the dawn of service-oriented architectures. As we are about to enter microservices land – before we define what microservices are by combining many ideas and concepts that we have already illustrated in the previous lectures – I want to have a more general discussion on what it means to change software, focusing on the most difficult part to change: the interfaces.

How do we change an API? We will see the most critical decision will be about whether to break or not to break clients. From the client point of view, it becomes critical to isolate clients and protect them from the impact of changes using layering. Layers give you a powerful design mechanism to contain the impact of changes. For example, every component is encapsulated by an interface. The interface can be seen as a layer mapping between the external data representation and the internal data representation. We will also take a look at how it is possible to design an open and extensible architecture with the use of plugins.

As we finally introduce microservices, we will see how it is possible to develop highly flexible architectures. In particular, we will discuss the feature toggle mechanism and a procedure to gradually migrate monolithic architectures and split them into microservices.

# API Evolution



How do we evolve interfaces? You can see here a physical example which shows you how hardware technology has evolved to feature very different kinds of ports available on different type of devices. We can spot a trend: over the years the set of ports has been shrinking as well as their diversity has been significantly reduced. One driver behind this change has been this idea that laptops should become thinner.

Another observation is that we can abstract and generalize the type of interface. Each USB C connector can play the role of any other port of the previous interface, and can also be converted back to the previous interface by purchasing the corresponding dongle. What's most surprising is that you can also power the machine through the same connector which is used also to exchange data. This was pioneered on mobile phones, where the available surface which can be dedicated to ports is however much more limited.

This interface evolution occurred through decades of technological improvement. Beyond reducing thickness, courage and simplification, what are the drivers that caused it? Is it a sign of progress that you actually remove features and simplify the interface so much that you can design a future where only one type of plug will be sufficient?

While life would be much simpler if one could cleanly break away from past decisions. Of course such disruptive interface evolution will have a big impact if you try to connect your shiny new laptop with older devices. Your decision to break backwards compatibility can sometime lead to creating new business opportunities, like selling adapters as a solution to the problem you created. With the additional benefit that if something breaks, you can always blame the adapter and not the incompatible hardware.

How are the users of this type of system impacted? Will they just follow along and silently live with the consequences of these upgrades?

Who has the power to force change across an interface boundary? This requires a closer look at the relationship between the two sides of an interface. Do you have a collaboration type of relationship where both parties are trying to build different components that have to match based on previously negotiated agreements? Or as represented in the picture: do you have an imbalanced relationship where one side is in control while the captive side will need to deal with the consequences of whatever decisions you take?

## Only one chance...

...to get the design right:

- Application Programming Interface
- Programming Language (and Standard Library)
- External Data Model:
  - File/Document Format
  - Database Schema
  - Wire/Message Representation Format

Once the API becomes public, it is out of your hands and it will be very expensive to change!

Why is it so critical to get the design of an interface right? Because you only have one chance to get it right. Once you have designed your API, then other developers will build applications on top of it. There will be hopefully many programs written on top of the platform that you provide for them. If the platform is not stable or poorly designed, the life of many developers will be a misery, unless they simply refuse to get near it.

This is the same position in which you find yourself if you invent a new programming language. I am sure some of you in your future career will invent a programming language or a domain-specific language. As you release the language specification, you release the compiler and maybe also the standard library that usually goes with it, it will be out of your hands. As other developers will try to write code in your language, they will notice if you make a change to the language as they probably will have to throw away all the code they have been writing and start from scratch.

We also have this type of situation when we design an external data representation: for example, the way data is represented as it is exchanged through an interface; but also documents written in a certain format; or data persisted in a database following a given schema. In general, we decide which would be the syntax, structure and semantics of the messages that we exchange between the outside world and our system.



These decisions are going to affect all the external entities that will ever interact with our system. All components that will ever have to read or write data that we can understand. And if all of a sudden you make a change to the format, every other component that used to be able to exchange data with us will be affected.

So once you release the API, then as a designer you are no longer fully in control because you share it with other people who will depend on it.

While before the first release, you could iterate quickly and do whatever modification and improvement you wanted, once it's released this will become a much slower process and also much more expensive in case you make a mistake, it will be super expensive or even impossible to fix it. If you made a small mistake in a programming language which eventually became highly used, people will still be laughing about it 25 years later.

## API Evolution

- Once in the API, keep it there forever
  - Never add something you do not intend to keep forever
  - Easier to add than to remove
- Make sure you explicit version all changes (including documentation) pointing out incompatibilities
- Publish preview (0.x) versions of an API before freezing it to get early feedback from clients
- Rename a component if its API has changed too much so you can start the redesign from scratch without breaking old clients
- Keep changes backwards and forwards compatible

Is there a simple rule we can follow to guide us with the evolution of our API? Once a feature is in the API, the simplest thing to avoid breaking your clients is to keep it in the API as it is, unchanged, forever.

If you are in doubt, leave it out. Never add something you do not intend to keep in the API forever. It's much easier to add something later rather than to remove it, or to change it.

Evolution by addition has visible consequences in many APIs, languages or data formats which have been around for a long time. Every change which requires to remove a feature is hard. APIs hardly ever get simpler, cleaner. APIs are rarely refactored. For example, if you discover a better name for a feature, you can easily introduce the new name, but you may only deprecate the old name as removing it completely will break clients. Instead, after each iteration, each new release, APIs just keep growing as they accumulate new features. Since nothing should be removed, the size and complexity of old APIs tends to grow.

Another important point is that there needs to be a clear specification of the interface. If there is a place where you have to use version control is exactly in this documentation of the API. This will help clients realize whether the new version becomes incompatible.

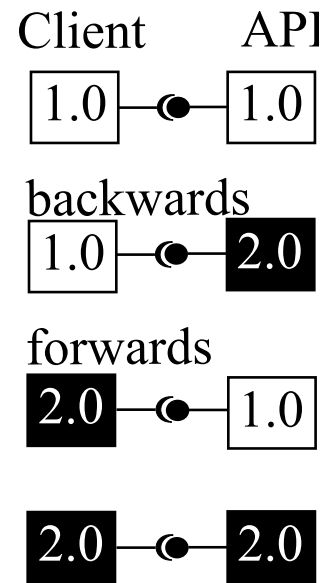
Every change that you introduce, you need to predict if this change breaks or not breaks clients depending on the previous version of the interface. There are some conventions that you can follow, for example using preview or experimental releases. So this means that you are going to tell your clients that this version is not stable. The API is not yet frozen. Only later the frozen API can be expected not to change anymore.

This helps to get feedback from interested clients: Is this feature useful? How can we improve it? But you still warn people not to seriously depend on it because it may still change. This is how you can avoid this one shot release that is very hard to get right because you only have one chance. To converge towards the perfect API design you need multiple iterations with clients that are willing and brave enough to depend on an interface which is not yet stable. It is only fair to warn clients that the API is not finished and may end up in a completely different shape.

If any API has evolved so much that you cannot recognize it anymore due to the amount of new and deprecated features that have encrusted over the years, it is better at a certain point to mark it as obsolete and start from scratch. Give it – this is very important – a new name so that clients do not confuse it with the previous one and somehow still expect it to be compatible. Especially if it's going to be totally different, you might as well change its name (not only the version identifier) as you make drastic changes to the interface.

# API Compatibility

- Backwards compatibility:  
new version of API  
compatible with old client
- Forwards compatibility:  
old version of API  
compatible with new client



What is important is to try to minimize breakage and to keep changes backwards and forwards compatible. Let's see what this means.

Compatibility requires that the interfaces match. Clients require something that is provided and there is a perfect fit so you don't need an adapter between the two interfaces.

Backwards and forwards compatibility use a spatial metaphor to describe how compatibility is affected by the interface evolution.

Each side of an interface can evolve independently. Here we can see that there is a client that depends on certain API and they're both in the same version. And then we see that this version evolves from 1.0 to version 2.0, so this is also a situation in which we can expect the two sides to be compatible.

However, there is a point in which, since we're talking about two different components it is possible to think that they evolve at their own speed. So in this case we have a scenario in which we have changed the API, we have changed the interface on which the component depends, but the component itself hasn't been touched. We have the old client talking to the new API. From the perspective of the API, we're looking backwards in time. We're trying to receive an incoming message that was sent from the past. If an older client works with a newer API it means that the change that you introduce in the API is backwards compatible.

How can you achieve such backwards compatible changes? The 2.0 version of the API is a super set of the 1.0 version. That means that when you plug the old client into it, the old API features they depend on are still there. So from the perspective of the old client, they will actually see the interface as being unchanged and will just ignore all the new features. That's easy to achieve if you are not touching what is already there.

Forwards compatibility refers to the opposite scenario in which the client is from the future and the API is from the past. So this is a bit more complicated to achieve because the client could expect to depend on features that the API doesn't provide yet. The question for you to think about is: how can you make an interface that not only works with today's clients, but also can still work with tomorrow's clients?

This was the definition of the difference between forwards and backwards compatibility. The term is defined from the perspective of the interface. If you go backwards, it means that the client is older than the interface, and if you go forward, the client is newer than the interface.

## Semantic Versioning

| MAJOR                       | . | MINOR                                                | . | PATCH                                             |
|-----------------------------|---|------------------------------------------------------|---|---------------------------------------------------|
| incompatible<br>API changes |   | new<br>functionality<br>but backwards-<br>compatible |   | backwards-<br>compatible<br>bug/security<br>fixes |

<https://semver.org/>

Additional versioning metadata:

- Build counter
- Release candidates

Once a version has been released, its content is **immutable**. Any changes must be released as a new version.

One way to identify the amount and the type of change that we have introduced in a certain element of the architecture is to use semantic versioning.

This convention uses three numbers: Major, Minor, and Patch. As opposed to floating point numbers, where there is only one dot, in computer science we use version numbers with two dots to identify how things evolve.

By convention when you increase the Patch number (on the right) this represents a small, internal change which can be assumed to be backwards compatible. Bumping up the patch counter typically occurs for fixes rather than new features. You might have a security fix. You might have a bug fix. Apart from the improvement, the system is feature equivalent as before.

If you make a minor change, this means that the system is growing with new features but still keeps the backwards compatibility. As long as the first number doesn't change, you can expect some degree of backwards compatibility as well as some improvement.

When you touch the major component of the version identifier, then you are signaling that there has been some incompatibility. This is a way to warn clients that they will not work without putting some effort to bring them up to speed.

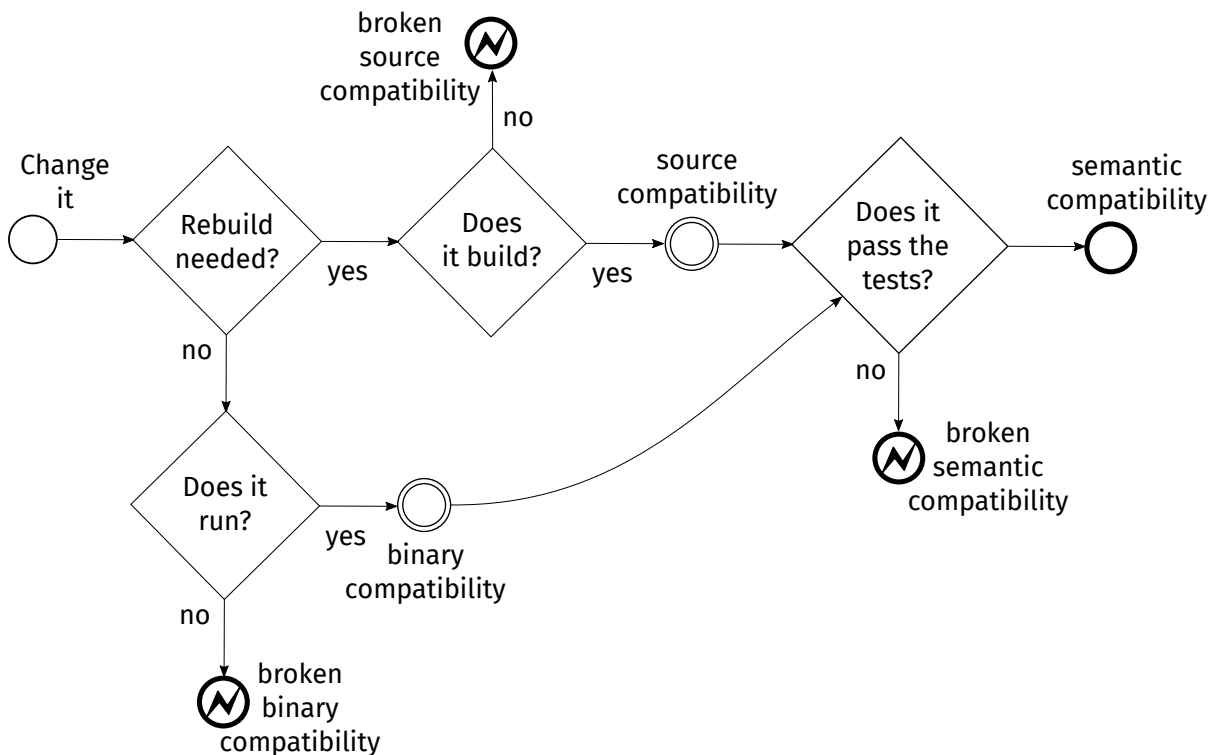
Sometimes you also include additional numbers: for example a build counter. While setting the major.minor.patch is something that you have to decide manually based on the impact of the change that you introduce, you can adjust the build counter automatically in your continuous integration pipeline. This is a very simple counter that is incre-

mented every time you run the pipeline, and this can be appended to the version identifier so that every build results in a unique identifier.

Sometimes as you are about to make a big jump, you know, while making a major release, you want to iterate quickly over it to polish and iron out the last showstoppers. To do so, you can make release candidates. This is version 1.0RC1. It is not yet stable, it is converging but it is meant to be tested and not for production. These RC version counter will eventually disappear once the version stabilizes and the release is accepted.

Version identifiers help to enforce a very important general rule: immutable releases. Anything produced from your automated build pipeline should be immutable and versioned. That means that this particular version that has a unique identifier corresponds to a known entity built using a certain reproducible process. Every version identifier corresponds to an explicitly tagged source that you have in your version control system. If you make any change to the source, you should run the build again and generate a new version. No matter what you do, no matter how small the change. That's why we have a build counter anyway. Even if you don't touch the other version counters, that build will increase and that will help you to guarantee this property, which helps to track the lineage of your release artifacts. Also, once you have generated the artifacts tagged with a certain version, you should never touch those manually. You should always recompile, regenerate, retest them through the automatic build pipeline.

# Changes and the Build Pipeline



Speaking about the build pipeline, let's see how different types of changes impact compatibility at different levels. Whenever we make a change we have to ask ourselves: Are the dependencies going to be affected by this change? In which way?

For example, if you change your library, do you have to rebuild your clients or not? If not, if the clients can run without rebuilding them, we just update the library and the clients just need to be rebooted to load the new version. If this works, then you have achieved binary compatibility. This is great because not only your clients are not affected, you don't have to recompile them, they still work even though you have changed their dependencies.

If this is not the case, you have broken binary compatibility. You will need to rebuild the clients. Clients need to be recompiled every time their dependencies change. After you have checked out the new version of the library, does the build still work? If not, you have also broken source compatibility: your client will need to be rewritten, to be extended, to be modified so that it can work with the updated dependency.

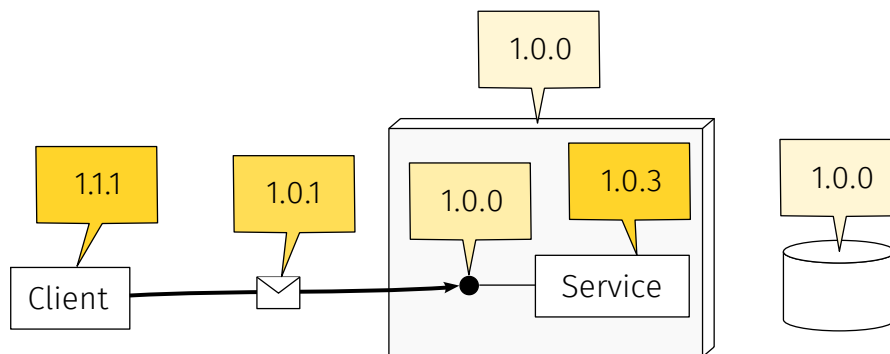
However, let's be optimistic: after pulling the updated dependencies, you rebuild and it works. This means you have source compatibility of the change.

After you compile, of course you should re-run the tests. This is true also when you have binary compatibility. After you update the library. You restart it, the new library links successfully, but will it pass the tests? If you pass the test with the change in the dependency then you have also a stronger level of compatibility. You have not broken the compatibility of the semantics of the implementation between the two sides.

If the tests do not pass, then even if you achieved source or binary compatibility, after all, your change has broken the actual semantics which the client expected from its dependencies.

You can use this flowchart to define this concept of binary vs. source or semantic compatibility. The three concepts fit with the major.minor.patch versioning scheme. Patching could be in some cases binary compatible, minor changes could still be kept source compatible, while if you have a major change you can expect also the semantic compatibility to be broken.

## Version Identifier



- Version Interface Descriptions
- Version Message Payloads
- Version Component Implementations
- Version Container Images and Addresses
- Version Database Schemas

After introducing version identifiers, let's see which elements of the architecture can be versioned. If you take it to the extreme, every architectural element can carry a version identifier.

For sure interface descriptions should be versioned. If there is an API that you depend on, this API is specified and there is a version identifier associated with it.

We can also version component implementations: the implementation behind the interface can have his own version. Also the client component on the other side has his own version. When a client looks up an interface, the directory will bind it to a certain implementation. Version identifiers associated with interfaces and implementations can be used by the client to constrain or identify which version does it depend on. When clients look up dependencies, they can look for the latest version, or a specific version within a certain range.

Once client interact with implementations, then the messages themselves, the messages that are exchanged can also carry their own version identifier. These are much more fine-grained than the whole interface. They can identify the version of the interface endpoint to which the message is directed. The message version identifier can also be used to check that it matches the version of the interface.

A more coarse-grained version identifier refers to the container or the image deployed

in a container. Images aggregates many components and interfaces which would need to be labeled so that they can be versioned as a whole.

When you start a given image on a container or runtime environment (whose configuration should be versioned), you will run a service on a certain server. The server also has network identifiers, for example, their IP address. IP addresses can also include version identifiers, for example if multiple versions of a service are active at the same time. Messages can be routed to their endpoints based on the version identifier embedded in the corresponding IP addresses.

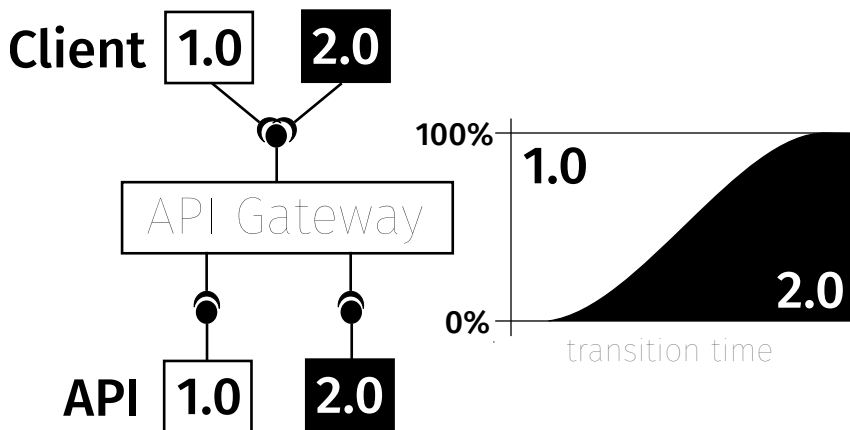
Finally, we also version the database schema used to store the state of stateful components. Why is versioning the schema important? Because if we change the structure of the data, we might need to do a migration from a certain version of the schema to another one. Therefore it's important to know what changed and the fact that we are going to connect our stateful component to a database which uses a different schema.

Ultimately, the goal of using version identifiers is to be able to have an easy, lightweight mechanism to spot differences. You can read an entire API description, read a schema specification and compare it against an older one. This is quite a lot of information and requires effort to spot the differences, not only by feeding them through an automated diff checker to detect which parts have changed but also what is the importance or the impact of such changes.

Or you can just read the metadata: there is an attribute that identifies the version and if the numbers change, you assume that they are different and depending on which number has changed then you will efficiently infer whether it's a minor or major change.



# Two in Production



Gradually transition clients to the new API version without breaking old ones

Once all clients have been migrated the old version can be retired

How can we use this notion of having different versions of interfaces and dependencies to support the gradual evolution of our system?

At some point when it grows beyond a certain size, it is no longer possible to apply a change in lockstep everywhere. If you have a very small system, you have the luxury to be able to decide that you make a change and everything changes atomically. As the system grows inevitably some elements fall behind. You will make a change on the server side and the client remains the old one for a while. Therefore it helps to be able to avoid breaking all clients and support their gradual transition over time.

If this transition takes time, it means that for a while you will have both old and new clients talking to your service, with your interface. How to keep them both compatible as cheaply as possible? Does this require any additional coding effort? It's just a matter of running side by side the old implementation for the old clients and the new implementation for the new clients.

You can directly connect the clients to the corresponding implementation. Or you can try to make it transparent and have a single endpoint. So you basically do not touch the address and then you use a special kind of load balancer that is going to route the requests depending on the version of the client. Each request message carries a version identifier which will help the load balancer to send it to the corresponding new or old implementation. It's important to put the version in the messages, because once you get a message coming from client 1.0, this component can forward it to the correct implementation.

If you have this setup to keep track of the traffic, you can keep track of how many clients are there which have been updated and they already use the new version as well as how many clients are still behind and they need to run against the old version. At the beginning, most of your clients are routed to the old interface. Over time you will notice more and more clients are shifting to the latest version.

This will also give you a chance to gain confidence that the new version is working. Most of your clients are still depending on the old which may be out of date. Maybe it doesn't have the features that you need, but at least you know that it's stable and it works.

And then you have some of the clients – the pioneers – going towards the future, they can work with the new system. As you gain more confidence and you put additional effort into migrating the clients then you will see that at the end you only have a few laggards, a few clients that are stuck in the old ways and eventually they disappear.

Once this happens, you notice that there is no more traffic going to the old version and you can switch it off and you just keep the new one. This pattern is called “two in production”. It allows you to buy the necessary time to perform this transition because there is a cost to migrate all the clients at once. Sometimes other people have to pay the price, but they still need to be willing to do it. And this way you can support them during this transition. The alternative would be to switch off the old version of the API immediately. Let the old clients break and while they are upgrading or unless they upgrade nothing will work for their site.

If you use two in production this doesn’t happen because the old still works. This means that you have to somehow give them an incentive to switch because of course if you keep supporting the old version forever then there is no incentive for them to do the migration.

Two in production is just a simple example. In reality in some architectures you have three in production: 1.0 will be the super old version that very few still use, but it still necessary to run it. 2.0 is the majority, that’s the stable version; the 3.0. is the experimental one about which you are starting to get feedback from the clients. You will soon want to introduce 4.0, but it’s too expensive to run four versions side by side and you will only do it when the 1.0 disappears. If you have three in production, you have a way to handle very old clients. The majority is in the center and then you have also a chance to test out new features oriented towards the future. So you have the past, the present and the future represented in your architecture. By keeping track of the traffic, this solution also helps you to know when you can discontinue the support for the past.

# API Sunrise and Sunset

Manage expectations and avoid surprises:  
warn clients that APIs they depend on are about to change and  
inform them about which features may change or disappear

|                             |                                                                            |
|-----------------------------|----------------------------------------------------------------------------|
| <b>Experimental Preview</b> | Still likely to change, feedback welcome, help give clients an early start |
| <b>Deprecation</b>          | Will no longer be supported in future releases                             |
| <b>Limited Lifetime</b>     | Will be supported during a limited time only                               |
| <b>Eternal Lifetime</b>     | Will you be able to keep the promise?                                      |

Regarding change and the life cycle of dependencies, it's important to manage the expectations of your clients. You want to give them a warning so that the APIs they depend on may change. And sometimes they even disappear. To visualize the beginning and the end of the life cycle, we can use the sunrise and sunset metaphors.

While change is positive because you grow the features, you make it more powerful, you improve the performance, you improve the availability, correct bugs: everything gets better and better all the time. But sometimes you also have to remove features that people need. Because it's too costly to maintain them. Your clients need to be warned that this will happen. To do so, we use these terms associated with different maturity levels of the interfaces and different kind of promises that we make towards the clients.

If you release an experimental preview, this means that your goal is to involve clients in the design of the API and still change it based on their feedback. There's a point in which you say, I think this will work. This is a good enough design. Let me see whether the clients agree so you open it up, you release it as an experimental preview. Some curious clients are willing to try it and give you feedback. Maybe you need to pay them for this. But in general their incentive is to get an early start with moving ahead with their side so that later when you make the final stable release, they already are well on their way to building something that can make use of it.

Of course, if during the experiment with the preview you discover that this will never work and you have to rewrite and redesign the whole thing, then of course they will be severely affected because whatever they invest into their client prototypes will also have to be thrown away. The goal is to minimize the chances this happens and have a good starting point in the first place.

That's how you start right at the beginning with every API. Every interface should go through such early feedback phase before you actually freeze it. Once you make a 1.0. it's frozen for good. But if you are at zero point something (0.99) then you know that you're in experimental preview mode.

On the other side, when you want to start to remove things, you should never make something disappear from one day to the next. You should first tag features that you plan not to support in the future as deprecated. If you write code there are standard annotations which you can use not only for the documentation, but even so that warnings will show up in the compiler logs whenever you are calling something that has been deprecated. Of course it's only a warning and can be ignored but you cannot say you have not been warned if eventually it disappears and your build breaks. You should have already stopped depending on it.

After this phase in which a feature that was stable becomes deprecated, then you can also add additional pressure to add: while it is "only" deprecated now, it will disappear within one year. This way, clients know they have one year of time to find an alternative to rely on due to the limited lifetime during which they can use what we offer.

End of Life is an important aspect of technology. Due to the constant churn of technology evolution, there are always better ways that are found, new discoveries, innovations, and therefore obsolete stuff to be thrown away in the wastebin of technology history. This evolution is also part of the business model: I sell you something that only works for three months, and then if you want to keep using it, you have to buy a new item that will be released right on time as the first one is expiring.

When they invented light bulbs, they made them so good that the light bulbs would never burn out. Then they realized they would soon run out of customers. So they actually designed the light bulbs to burn out after some amount of time, so that people will have to buy new ones to replace them. You can either know in advance when something will expire or you can have a surprise that makes your life interesting. Why did it have to really run out today?

Now, if you try to get a competitive edge, you can also try to release an interface and promise that this will be supported forever. This should make your API more attractive. If clients need to build a system which will depend on you, for them is much better if a dependency will be stable and supported forever, as opposed to having uncertain information about its lifetime or just a limited guarantee: we will support it for a couple of years, but then it will most probably change in a non-backwards compatible way. Clients will have to renew their license or sometime bribe us so that we can continue to support the old version for an even longer time. Or they look for the cheapest upgrade path, if they are unable to find a compatible replacement, in the worst case, clients will need to adapt and rewrite their code to fit with the new version.

# To break or not to break

- Sometimes it is unavoidable
- $\text{Cost}(\text{Keep Service Compatible}) > \text{Cost}(\text{Fix Broken Clients})$
- Warn clients in advance



On June 30, 2010 (9 weeks and 3 days from now), the [@twitterapi](#) team will be shutting off basic authentication on the Twitter API — you can see the announcement [here](#). This has been previously announced. All applications, by this date, need to switch to using OAuth. Don't fret! @twitterapi is here to help! Feel free to reach out to us directly, or via our Twitter Development Talk group.

- Provide migration path (to alternative provider) or fallback solutions (shims, polyfills)

As you consider retiring an APIs – depending on the languages that you use to describe it – you can annotate its feature with these terms: deprecation or sunset. These are mechanisms within the way that you describe interfaces (deprecation) or the way that you interact with them (sunset headers) to annotate them with this "expiration date" or "guaranteed to work until" metadata. This particular feature is deprecated or this particular endpoint will disappear within a certain time. Even when we sunset or deprecate APIs, callers will still come knocking!

The question is: why do these breaking changes happen? Sometimes it's something that is unavoidable. Maybe it's not even your fault, like when the break is caused by a dependency down the stack that is outside of your control. You wish you could maintain the compatibility and support this interface for longer, but then you discover that something else stops working and then as a consequence your component stops working in the way it used to and your client is broken.

There is a simple comparison you can make. Is the cost of keeping the service compatible and supporting all clients greater than the cost of fixing the clients that will be broken by stopping the compatibility? The cost of keeping the service compatible is paid by the service provider. Whereas the bill for fixing the broken clients is paid by the clients themselves.

What kind of relationship have you established with your customers? If you pay the price for maintaining compatibility, can you offset this additional cost to the customers themselves? If you are unable to maintain compatibility and you break them, then they will have to deal with the consequences. It may be cheaper for clients to avoid breaking changes by paying the service provider to maintain the service compatible as opposed to fix the clients.

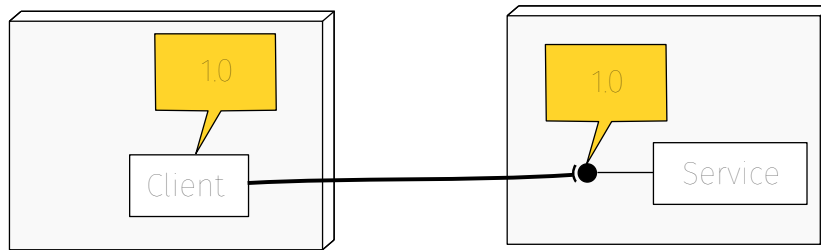
Also, if you're going to break it, don't do it in the middle of the night when nobody

is watching. Break it with an advance warning. This is an old example, it's called the "OAuthpocalypse": there was a Twitter API which changed on a fundamental level the way that users authenticated. And this, as a result, ended up breaking all of the apps and all of the clients trying to access the Twitter API. To warn developers (and users) they set up a website prominently featuring a big countdown clock. The message was clear: you have nine weeks left to update your application or to pressure its developers to release an update for it.

Despite these advanced warning, the day they flipped the switch, there were still many clients that stopped working. This is something related, in general, with how people react to deadlines.

Too bad they could not simply redirect the old clients to the new endpoints. Together with a warning, you should also provide a migration path. Or some fall back solution so that the client can adapt and avoid the worst possible outcome.

# Who should keep it compatible?



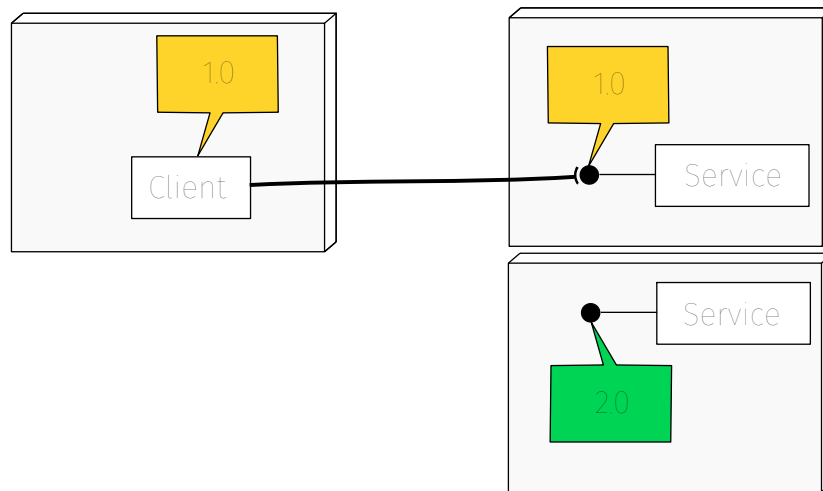
We've seen when you break compatibility, you introduce a mismatch between interfaces. A possible solution is an adapter, if we assume that there is an equivalent alternative way to do the same thing.

If a feature is removed from the new interface, the feature is gone and there is very little an adapter can do. Unless the missing feature gets re-implemented from scratch inside the adapter.

Let's take a look at the interplay between adaptation and evolution. This can also help us to explain more in detail who needs to pay the price to deal with the consequences of breaking changes.

Here we start – nice and easy – from where everything is compatible.

# Who should keep it compatible?



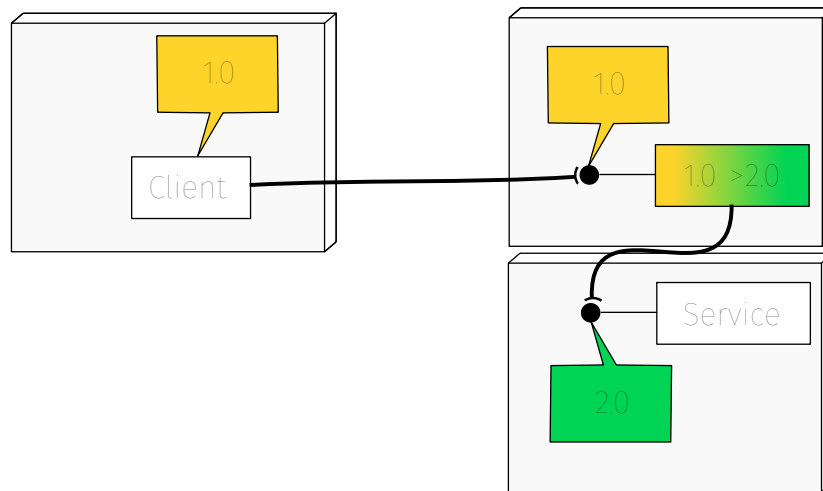
## Two in Production: Client still using old version

Now we introduce a change. There is a new version of the interface. Although there is a new version, we decided that we're not going to stop the old one, because otherwise the old client will break.

To avoid that we are using two in production: two versions of the service are running side by side and the client is still depending on the old one. With two in production, the advantage is that the investment in the old version of the services is already amortized and we keep running it as it originally was. Basically, we don't touch it. Next there is a new version of the service that has just been released. Freshly developed, maybe after a complete rewrite from scratch, or just some changes which unfortunately break compatibility.



# Who should keep it compatible?



Provider keeps supporting old clients by running the adapter

As it is running on the side, conceptually, we could use an adapter to replace the old implementation and switch to the new implementation while keeping the interface compatible with the old version, which is the one required by the old clients.

For example, such adapter may need to fill in default values for data fields which were added in the new version, or rename a few fields from the old (and deprecated) names to the new ones, which offer an equivalent semantics.

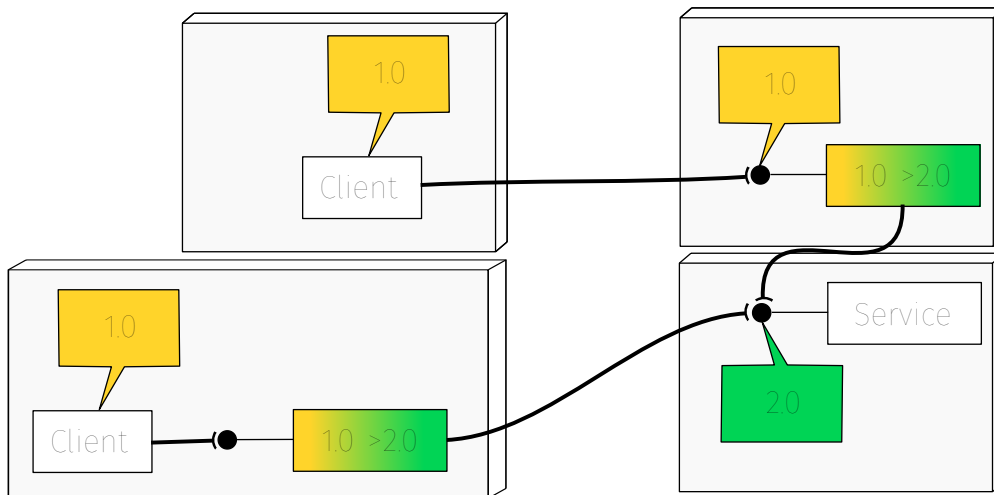
In general, if it's possible to implement it using an adapter, the old clients will not notice it as they still think that they are using the old version. Behind the old interface, the adapter just forwards their calls to the new version by transforming them as necessary.

Why would you do this as opposed to keeping two in production? What if your service is stateful? If you run both versions in parallel, their state will drift apart. The state of the old service is isolated from the state of the new version: if the client makes a change to the data through the old API, this data will not be visible from the new interface, because it's a completely separate system.

If you can merge the two versions (or at least their state) in this way, the old client and the new client will be able to see the same state of the service, each from a compatible interface. If you can manage to have a single database where you keep the state for both interfaces, then, for example, a new client can write some information into the system through the new interface and then you can read the same information from the old client using the original version of the interface.

Still, the cost of running this adapter is something that the service provider is paying for at the moment. Because we are still offering two versions of the interface, we have just merged the underlying implementation and state, which are all running on the service provider side.

# Who should keep it compatible?



Old clients need to run the adapter to use the new service version

We have reached the point in which the service provider is no longer willing to pay the price of maintaining the old version of the interface and running the corresponding adapter. While there is a way to do the adaptation, the service is no longer motivated or interested to perform it for its old clients, whose population may have been steadily disappearing. The client should actually upgrade and use the new version, but what if the client cannot do that. What if the client is stuck in the stone age?

It may happen that clients are unable to upgrade due to their obsolete hardware/operating system version. Or simply their source code was lost or there are no developers left on the team capable of correctly modifying their code.

In this case, it is still possible to keep the adaptation running but just push it onto the client.

From the service provider perspective, only the version 2.0 is now in production. Old clients have to install and run a local adapter in between so that their old client can access the new service interface through the adapter deployed on the client side.

This is what happens when you hear about installing a piece of software called a “shim” or a “polyfill”: you have an old client, you augment it so that it can use the new service, the new version of the API.

# Layers

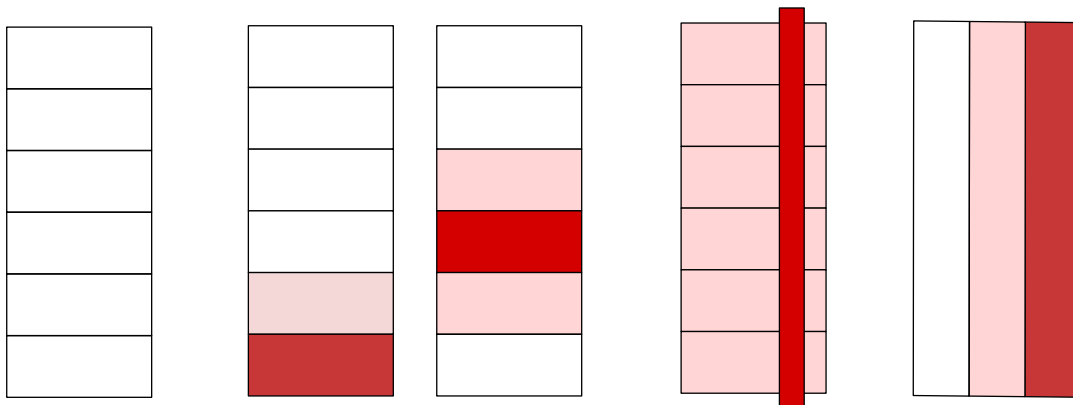
How to contain the impact of change?

- Change of Dependencies
- Change of Technology Platforms
- Change of External Service Provider APIs
- Change of Data Representation Formats

What if we expect changes in our dependencies? Something that we depend on will change. We can have changes in the dependencies underlying the system, the technological platforms on which we build it tend to slowly evolve underneath. Also there can be sudden changes from service provider APIs. Likewise, data sources move, data formats shift and we have to avoid coupling too tightly the data that we use within our system with the external one.

How can we control and limit the impact of changes? Add a layer of indirection.

# Layers



Changes of one layer only affect the neighbours

Cross-layer changes may impact all layers

The simplest idea to isolate the impact of changes is to use this concept of architectural layers.

Consider this layered architecture. What if we change the bottom layer? Which layer is affected? Only the one next to it. If I tell you the changes are happening at the top layer, then you probably see that the layer that is affected is only the one right underneath. If you change something in the middle, this will probably affect both sides: these layers are directly in contact with the one that is affected. It may also be possible to constrain the impact only on one side, depending on the dependency relationship direction between the components of each layer.

There can be layers where the impact of changes goes only in one direction. For example, from the bottom to the top. If you change something on top, you don't affect what is underneath. This helps to constrain the impact of changes. But if we change fundamental layers at the bottom, also in this case if layers are done right, the change will impact only the immediate layer on top and should not ripple further up the stack.

There are some changes however, which unfortunately happen to impact all the layers of your architecture. Depending on the strategy you followed to introduce layering, it will be difficult to anticipate all possible changes and just keep them isolated within certain layers. So-called cross-cutting changes will impact all layers of the architecture.

While usually you can change the implementation, for example, optimize its performance, without touching the interface so that clients above are not affected, what if you need to add support for a new feature?

This new feature will require an extension of the implementation. It will also need to be exposed through the interface. Likewise, the component above will need to be extended so that it can make use of the new feature. Ultimately this will ripple all the way up to the user interface so that users can actually see and interact with the new feature. Also, the change will sink through the layers all the way to the database because the new feature needs information from the user and this information has to be stored somewhere.

For such an extension, for such a change, your layering is completely useless because you have to start from the top of the system, add a new format, the new UI widget, extend

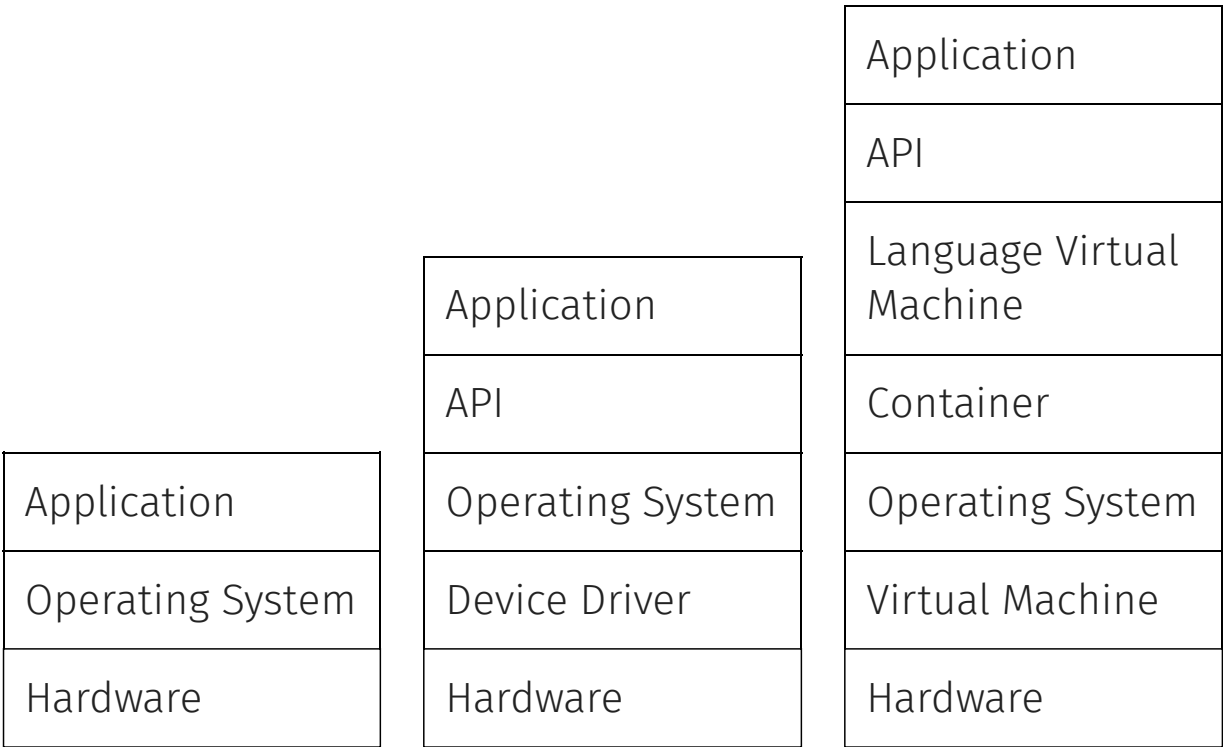
the API, extend the business logic controllers, extend the model, make the new information persistent, all the way to the bottom.

If you're planning to grow the system in this way, you chose the wrong layering strategy. Maybe your goal should have been like this to slice the architecture in vertical layers. This way, your changes, your new features are encapsulated and the impact of adding them stays within one of those vertical sections and the others are minimally affected.

Depending on the level of granularity, depending on the goal of your decomposition strategy, you should not feel constrained to use only one kind of layering. If you zoom out far enough layers will appear to be oriented vertically, while if you zoom into each of the vertical slices, horizontal layers will appear.

The main concept remains valid: use layers to surround components likely to change to control the impact of such changes on the rest of the system.

# Layers



Let’s take a look at some examples. The classical layers: application, operating system, hardware. The OS layer isolates the application from as well as it abstracts the hardware on which the application is running.

Within these layers we can add more intermediate layers. We can add the operating system API which hides the implementation from the application. There is a clear point in which the application touches on the operating system. The operating system can change underneath without affecting the application, as long as the API remains stable.

We also have device drivers to abstract the low-level hardware and separate them from higher levels of the operating system. Again, originally we have the operating system on top of the hardware. More in detail, there is an additional layer in the middle that will need to change if you change the hardware, but as long as the driver interface (e.g., for a generic class of devices) remains the same, the rest of the operating system is not affected.

If we introduce virtualization, we add a few more layers. There are virtual machines for specific programming languages that are running inside containers. There are guest operating systems that run inside virtual machines. And you can have as many layers as you want, so that if you make a change underneath what is above is not affected.

While we have seen such abstraction layers when we were discussing portability, here we emphasize their benefits concerning the evolution of the system.

Users want to be able to upgrade the operating system to get performance improvements, apply security patches and so on, but users do not want their applications to be affected. If every time you ship an upgrade of the operating system you break all the applications, many users will be annoyed. Designing appropriate interface layers helps to deal with this backwards compatibility requirement.

# Layers

|              |                          |
|--------------|--------------------------|
|              | User Interface           |
| Presentation | API                      |
| Logic        | Business Logic           |
| State        | Object-Relational Mapper |
|              | Database                 |

Another classical example is the layering of an architecture into: presentation for the user interface interaction, the business logic, and then we have the persistence layer where we separately store and manage the information processed by the application. The persistence layer deals with consistency, concurrency control, optimized query planning, and all the features that you get from a database.

Why does this layering help? If you make a change to the user interface because you switch from using a command line terminal to a graphical user interface where you can use the mouse to select menus, click on buttons and dialog boxes, and then you switch to a touch based user interface, then you switch one more time to voice control but then augmented or virtual reality becomes fashionable.

These are all superficial changes affecting the user interface of your system. They should not affect the underlying logic nor the information content, only its visual representation.

You can focus the impact of the user interface technology evolution on your system in one place (the user interface layer) so that the rest can remain the same.

We can also zoom in so we can see that to isolate the business logic from the user interface we can introduce again an API. We make an explicit interface. Likewise to isolate the business logic from the data storage we can have a dedicated mapping layer, like for example an object relational mapper. This layer is going to take the memory data structures that we use when we process the information and transform them so they can be stored persistently. As a result, the business logic becomes independent from the database schema. The representation that we use to store the data does not need to be the same one that we use to process it.

# Layers

|                                                    |
|----------------------------------------------------|
| External Data Representation                       |
| Validation, Mapping: Serialization/Deserialization |
| Internal Data Representation                       |

Generalizing from the previous case, as a last example, we can also distinguish “Data on the Outside versus Data on the Inside”.

The external representation for the information can be different from the internal one, and this can happen every time you cross any component boundary, any interface in your architecture. Each component has an inside and an outside. Any information that is traveling between components will cross such boundaries. That is, the external data representation is used to exchange information between components, which however may store it as part of the state of the component using the internal representation, which is local to each component.

You have the opportunity as you cross the boundary of the component to transform between these two representations.

This allows you to couple the implementation of the component to the internal representation, which has a local scope so that the implementation can evolve without being coupled to the assumptions that you make on how the data is represented outside.

If you don’t take advantage of this opportunity, it means that every time you modify your data (e.g., how it is represented, its syntax, structure and semantics) somewhere in your architecture, every component will be affected. Every component will need to know that the globally defined and universally shared data types have changed.

For example, the data structure to store the customer phone numbers has been updated, so you have to rebuild all the components that deal with customer data. If you can keep this change local within the internal data representation of one component, then you keep components compatible and need to rebuild only one component.

Going from external to internal and vice versa, we find a mapping layer sandwiched in between. This layer is responsible for serializing the internal data so that you can send as the payload of an outgoing message, or deserializing data when you receive the message. With every interaction, messages need to be received, buffered, read, parsed so that the information needed by the component can be extracted and processed internally.

How much validation to do before or after this transformation? As the data enters (or leaves) the boundary, there is an opportunity to check that what arrived from the outside is correct. Ensure that it makes sense before you transform it into a suitable internal representation. The layer not only builds a line of defense for design-time coupling between data models but also can be used at run-time to detect illegal border crossings of invalid messages.

The layers shown in this example help to reduce coupling because when change hap-



pens on either side, the only place that will be affected is the mapping layer. If the external data representation has changed, adapting the mapping makes it possible to keep the same internal one. Everything else within the component that depends on it will not be affected. This is possible thanks to mapping layer which absorbs this change. Likewise, if you want to change the way that you represent the data within your component, the rest of the architecture is not affected because the mapping is going to transform the data on the inside to the shared standard representation visible on the outside.

# Tolerant Reader

*How can clients or services function properly when receiving some unexpected content?*

Ignore unexpected content and fail only if required content is missing

Tolerant Readers ignore new items, the absence of optional items, and unexpected data values for which defaults can be used instead

Every time data is exchanged through the interface there is an opportunity to perform validation, especially for what concerns external messages that arrive into your component. Before accepting them, one should check them so that if the message goes past the interface the rest of the system can make some assumptions about the content of the message and will not have to worry that this might be an invalid or incorrect message.

When we do this validation we have two options: to be strict or to be lenient.

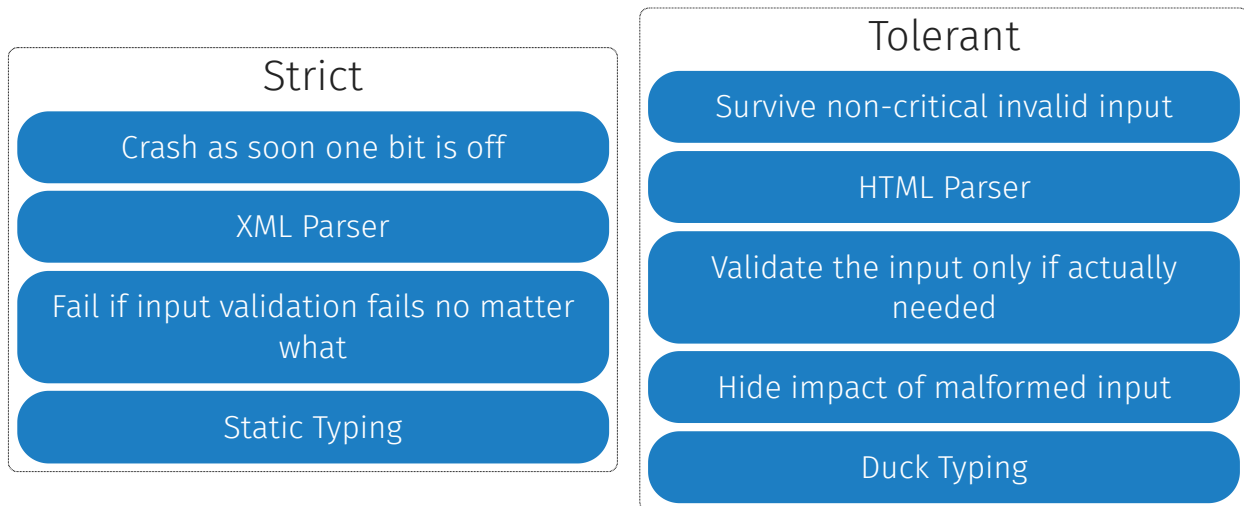
How can clients or services function properly when receiving some unexpected content from the outside? The idea of introducing a tolerant reader is to ignore the unexpected content and only reject the incoming message if content that is actually required is invalid or missing.

As opposed to saying that this message has one bit that doesn't fit with my expectations, therefore I reject everything. If the bit that you didn't expect is not something that you actually depend on, then you can survive by ignoring it and just reading out what you really need.

As we introduce a change in the structure of the messages that we exchange, and in particular we add something new. We may wonder about whether the change is backwards and forwards compatible. If we have a tolerant reader, it is much easier to keep the compatibility: any unexpected items (including the new ones) will be ignored. Also if you have optional items that are missing, they will be ignored as long as the reader can fall back to read a default value. Any value that is missing as well as out of range can be reverted back to a default so that the tolerant reader can accept the input.

# Tolerant Reader

## Which kind of reader?



Which type of behavior fits with the definition of tolerant reader? and which one is the behavior of a strict reader?

A strict reader will crash as soon as one bit is off. And will fail if the input validation fails no matter what, whereas the advantage of a tolerant reader is that it will only validate the input which is needed. If something that is not needed is invalid, it will still accept the message for downstream processing.

Attempting to hide the impact of malformed input can be seen as a good recipe for being flexible and surviving the impact of change. It can also be a problem in case you are trying to estimate or detect the impact of a change, because if you hide the impact of such problematic changes then it will actually be more difficult to discover the original source of such malformed input.

We can see two representative languages (HTML and XML): the behavior of the corresponding parsers is a good example of each type of these readers. As most beginner Web developers, you may have experienced that web browser is very forgiving when reading strangely formatted HTML pages. Instead, if you try to parse XML messages, the same kind of errors will not be tolerated. As soon as you forget to correctly nest one tag, then the whole document will be rejected.

You can also think of the difference between languages that are strongly typed and languages which are dynamically typed. You notice the difference when you try to take a JSON document and you want to parse it into a Java object. If the structure of the JSON document doesn't fit your class definition, then there will be a problem to transform the external representation, which uses a flexible representation format, into the internal one which is much more constrained.



# Design for the future, because it will be here sooner than you think.

Let's think about the future because the future will be here sooner than you think.

How does the future influence the design of our architecture?

There is a tension between your expectations about what the future will bring and whether you should already act on them in the current version of your architecture.

We have already seen in a past lecture the term YAGNI (you are not going to need it) and how this influences API design. Still, if there are some things that you can anticipate that you will most probably need, then you might as well get your architecture ready for supporting them in a future version.

If you might need it, you should prepare for it. You don't have to build it yet, but it's a good idea to ensure your architecture is extensible so that these future needs can be accommodated without having to redesign and rewrite the whole system from scratch.

# Extensibility and Plugins

*How to design an extensible architecture?*

use plugins

Allow end-users to independently choose which components to deploy anytime as long as they fit with the extension point interface

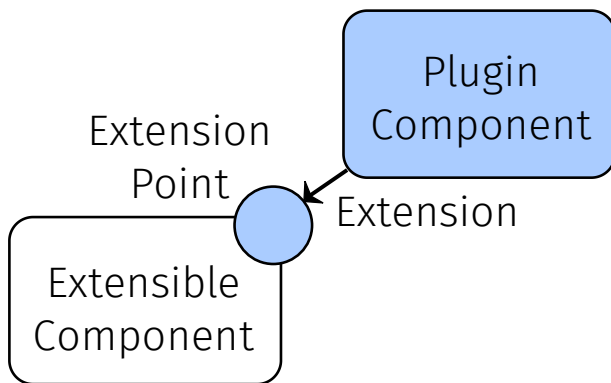
Plugins are a particularly useful mechanism to design an open and extensible architecture which can grow in anticipated ways.

With plugins, the choice of the components that make up the architecture of your system is pushed and delegated all the way to the end users who are going to deploy and run your architecture. The end users will choose which components – in this case we call them plugins – they are going to configure your architecture to load, either on startup or while it runs.

How do you control which components can be used? You can only use compatibility as a constraint. So if users try to plug in something that is not compatible, clearly the plugin is not going to work. But as long as it fits, the user can plug in any component they want into your architecture.

Introducing a plugin system opens up your design of a highly flexible and extensible system. It is also risky because the more the architecture can be extended by third parties, the more you are willing to accept code developed by someone else. When something goes wrong, you don't know exactly who to blame. Adding unknown plugins from random sources may void the warranty. If users attempt to extend your system in unsupported ways, they breach your support contract. Not to mention if untrusted plugins act as trojan attack vectors.

# Extensibility and Plugins



The architecture defines explicit extension points so that the system can be customized or even dynamically composed out of plugins

Examples: Browser Extensions, Eclipse, Photoshop, WordPress

How does the plugin architecture work?

We are still in the context of a component-based architecture in which however you define a specific type of interface which is called the “extension point”. This makes it possible to design extensible components whose interfaces contain explicit extension points in which other components can be connected (or plugged in) as long as they match the assumption and the expectation encoded in the extension point interface.

We see this type of extensible architecture everywhere. Web browsers can be easily extended and customized by downloading and installing so-called browser extensions. Another example is the Eclipse technology platform where the whole development environment is actually built out of plugins.

Opening up your system with plugins also can provide support for a business model where you make money not because you sell the platform, but because you sell plugins. Like Photoshop filters. Like WordPress, a content management system which is also the foundation for a rich ecosystem of all kinds of extensions.

# Extensibility and Plugins

Pre-requisite: Plugin interface description and discovery

Mechanism: Plug in and plug out

When are plugins plugged in?

- Static (configuration/deployment/installation)
- Startup
- Dynamic (without restarting the component)

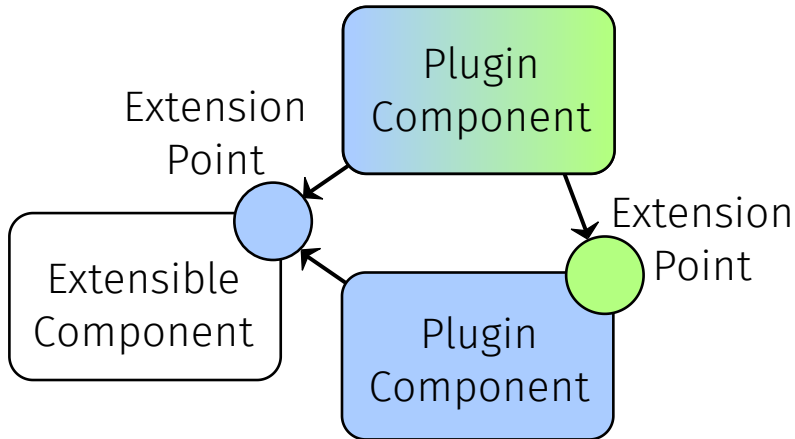
Ecosystem: the extensible component becomes a platform where alternative plugins compete

When are plugins initialized or selected? You can extend an architecture statically when you install the system. As part of its configuration you can refer to a number of pre-deployed plugins, which will be loaded when the system starts. Configuring which plugins should be loaded can be as simple as storing the corresponding artifacts or packages in a given folder. Still, unless you stop and restart it, you will not be able to change which plugins get loaded and activated at runtime.

We also have the option of a dynamic plugin system where the linkage between the plugins and the base components is actually done at runtime. What's interesting is that you can plug in and also in some cases plug out components without having to restart the whole architecture.

# Extensibility and Plugins

Cardinality: Multiple Plugins can fit in the same extension points



Recursive: Plugins get extended by other plugins

In the simplest case, we have one component that is the foundation which can be extended by plugins. This design can be applied recursively. Not only we can have an extension point in which we plug multiple plugins, but we can also recursively plug plugins into plugins. Plugins themselves, in other words, can be designed to be extensible and offer their own extension points.

We have seen how to achieve extensibility with plugins and extension points. These are solutions to design flexible architectures from the days of component-based software engineering. Plugins assume that you physically download and install the components to run them. As you do so, you can also download, configure and activate additional plugins. Users enjoy the flexibility of this kind of open system because they can decide how to extend it long after the original release has been shipped.



# Microservices

The microservice architectural style is an approach to developing a single application as a suite of **small** services, each running in its own **container** and communicating with lightweight mechanisms, often an **HTTP resource API**. These services are built around business capabilities and **independently deployable by fully automated deployment machinery**. There is a bare minimum of centralized management of these services, which may be written in **different programming languages** and use **different data storage technologies**.

We have already discussed the evolution of components into services because of the impact of the Internet on the software industry. While services focus on availability and successful services also need scalability, what about flexibility?

We have now reached the need for the ultimate level of flexibility in which we not only deliver the software as a service, but we do so in a way that it can be rapidly evolve and it is easy to change.

Here you can read the classical microservice definition, in which we can see microservices positioned as an architectural style: an approach to develop a single application decomposed into multiple small services. So the definition builds on the notion of service, as long as their size can be measured and be kept small.

Each of these microservices is deployed and runs in his own container. We already discussed what containers are. Many of the concepts we have previously seen are coming back. Each runs in a dedicated container so that it can have an independent operations lifecycle.

To communicate, microservices use some lightweight connector mechanisms. For example, some API based on HTTP. So this makes it possible for them to be deployed remotely. They don't have to be all running in the same local environment, because they can send to each other HTTP messages.

Then there is the sentence that describes how to decompose your architecture into these microservices? Each of them should do something specific from a business point of view. Each implements a different feature. Each delivers a different business capability. Somehow these design heuristics should guide you in your quest to decompose them and select which service you will need.

What is most important is that they are independently deployable. If each can be deployed independently, your architecture is flexible. You can change a microservice and you can deliver the change as soon as possible. You can release it, you can install it and you can run it all independently of the other microservice that you have in your architecture. This is fundamental and if it is not true for your architecture, then you can hardly call its components microservices.

Since such deployment occurs often and you need to do it independently for many microservices, you shall do it supported by some automated deployment machinery. Yes,

we already have discussed how this works during the lecture on deployability with continuous build, integration, delivery and deployment pipelines.

You should also try to keep centralized management to a minimum. Every team that you have is dedicated to an independent microservice and they need to coordinate only concerning the interfaces, the microservice APIs. Regarding taking all other kinds of decisions about how the microservice are implemented, for example, the choice of programming language or the decision on: how are we going to store the data? Which database should manage the state of our microservice? All of these decisions can be taken independently.

Why is this important? There are many different programming languages. There are many different storage technologies and you want to pick the best solution for the problem that you have. And it's not true that if you use the same database, you will get the same performance or the same expressiveness that you need to represent information in the appropriate way for all microservices. The idea of using microservices is that you can choose and pick the right technology for the job without any centralized constraint.

The only constraint which needs to be in place concerns the interfaces and the connectors. If you want software written in different programming languages to be deployed you can use Docker and if these heterogeneous components need to talk to each other, they can speak HTTP. If you want to reliably change them, you use all the automated deployment machinery (containers) and to make it fast to evolve them, you must keep their size small. Or at least, that's according to this classical definition.

# Monolith vs. Microservices



To share with you a visual image of the elements that we are going to discuss in this lecture, I would like to introduce to you the Monolith.

The Big Monolith at the bottom represents the situation in which you do not want to be. If you make a change to the monolith, you will do so slowly because it just takes too much time and effort before you can make a new release for the whole thing. It will also be very expensive to deploy it in the Cloud.

What we do is break it down into microservices, which are supposed to be small. Here on top of the monolith you can see a couple of stacks of microservices. They are so lightweight and easy to deploy, for example, as a cloud native application. If one stops working, you can just throw it away and cut a better one.

# Will this component always terminate?

```
function f() {  
  ...  
  return 42;  
}
```

## Development

To understand how architectural abstractions have evolved all the way to microservices, let's take another look at components.

The main concern when developing software components is whether they are correct: whether they give correct and timely results. Thanks to their interfaces, components become reusable. They can also be interoperable thanks to standardized interfaces. We can try to minimize their dependencies, and ensure they are self-contained.

The most challenging part of building high quality components is to prove that their implementation is correct and just in case also write some tests to highlight the presence of some bugs in case regressions may occur after releasing bug fixes.

You definitely want to avoid releasing components which when invoked will take forever to reply.

# Will this service run forever?

```
while (true) {  
  on f {  
    return f()  
  };  
}
```

## Operations

When you move to the cloud, when you publish your components on the Internet, then you have another issue which is no longer to make sure that the component terminates but it is actually the opposite. When a software component is delivered as a service, you must ensure that it is provided with an acceptable level of availability.

This means that the service has to run forever.

You have to do whatever it takes to make sure that the service never stops. While the service is running, clients will call the service and its developers then inherit the previous challenge. The service has to be able to compute something and give an answer as fast as possible, no matter how many concurrent clients are there.

We have discussed the concerns of scalability and availability before. These are quality attributes achieved through operations. We've seen that you can introduce redundancy into the architecture. You have to monitor heartbeats, you run a watchdog which will take swift recovery actions when something fails to recover. Likewise, you will run an auto-scaling controller which can allocate or deallocate resources depending on the workload.

Architects, developers and operators learn how to design, build and run service oriented architectures which have to be available no matter what. And in some cases, if they are successful, they also need to scale.

# Will this microservice continuously change?

```
while (true) {  
  on f {  
-    return f()  
+    //return f()  
+    return f2()  
  };  
}
```

## DevOps

As we evolve into microservices, we have another challenge ahead. We not only have to build something that runs correctly and gives back a result; not only we have to keep it available and possibly scale it, but we also have to be able to continuously change it.

The only way to be able to do all of the above requires developers and operators to work together and close the software lifecycle. This is why it is called the development and operation (DevOps) lifecycle. There's no longer a boundary. There is no distinction between the developers and the operations anymore since we are continuously changing and redeploying our system.

It's easier to do that if you keep the amount of changes that you do and the size of the overall component as small as possible.

To summarize the differences between components and services: components are reusable; they need to be deployed and installed and they are operated under the ownership and control of who is using them. So typically you build a component, you release it. And then you can forget about it, as it runs outside of your control.

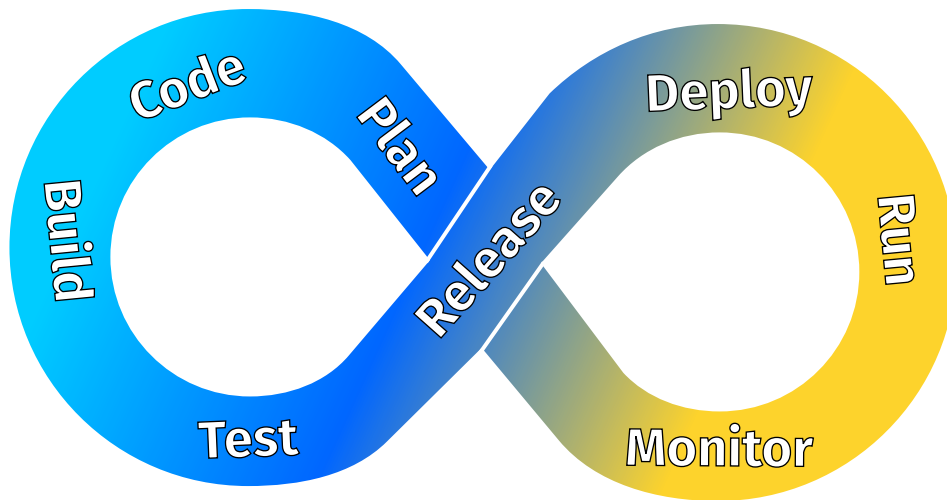
If we make the component remotely accessible, to also give quality of service guarantees about its availability we need to be in charge of running it, not only of developing it. So we inherit all challenges component developer face, but also we become an operator tasked to keep the service alive so that users can call it from all over the world.

If we can do all of that but also keep continuously evolving our system, then we have a microservice architecture.

So I hope this has clarified the core differences between software components, services and microservices. Do not worry too much about the size. We will see that it is not so important. What is critical is this ability to continuously change all the elements of your architecture in a way that doesn't limit the rate of change and can push it to hundreds of releases per day.



# DevOps



Close the Feedback Loop between Operations and Development to significantly speed up the development release cycle

Scripting of Automated Build, Continuous Integration, Testing, Release, Deployment and Operations Management tasks

We have seen this DevOps cycle already. While before we focused on the deployment transition between development and operation, with microservices we close the loop and run it as rapidly as possible.

To do so, you need automation, you introduce continuous integration, build pipelines, you ensure quality with tests. If you can make sure that making a release and a new deployment is as easy as clicking a button. If there are no risks to try out a new version with green/blue deployments, if doing experiments for A/B testing is not a major multi-year, high-risk project but something common, which happens every day, then you are already going through this cycle multiple times.

Adopting microservices implies adopting this type of practices and all the technologies (automation, continuous build pipelines, containers) that go with them. The goal is to be faster in doing iterations over the lifecycle. And if you can iterate faster, you can be faster in improving your system as well as learning from your users by observing them, getting explicit feedback and dealing with it.

Also you can afford to move fast and break things. It will happen: you will make mistakes, break things, but as fast as you break them you should also fix them. We have seen you should be able to undo failed releases. Also with two in production you should not have to expose all clients to a new service version.

Overall, with devops the goal is to move fast while keeping the quality high. Quality and speed are no longer a trade off, with the appropriate safety net of practices and infrastructure.

## DevOps Requirements

Confidence in the code (code reviews, automated tests, small, incremental releases, feature toggles)

100% automated, dependable and repeatable deployment

Monitoring feedback (smoke tests, A/B testing, analytics)

Elastic runtime (dynamically adapt to workload changes)

Secure build and runtime (every change may introduce weaknesses)

How can you check if you are ready to adopt microservices? Are you in a company that not only wants to adopt this architectural style but also is willing to follow these continuous development and release practices?

First, you need to have confidence in the quality of what you do. There are many techniques which help boost your confidence. For example, you want to adopt code reviews. I hope you have been hearing about what a code review is in some of the software engineering lectures. You should also have automatic quality assurance. This requires to write, run and maintain all kinds of tests: unit tests, integration tests, capacity tests, performance and scalability tests. Regarding availability, you can use chaos engineering to turn unlikely or infrequent events into regularly occurring ones: inject failures in a controlled way and see if your system survives thanks to the automated recovery tools you have introduced.

The larger your code, the more difficult it is to be confident about whether it meets its functional and extra-functional requirements. Here is the microservice size argument again: changes should be small so that their impact is easier to control. Also you want to make frequent incremental releases. You do not want to make a Big Bang release every year. You want to make continuous releases every few seconds. There is one technique that we're going to discuss that helps you to do that is called feature toggles.

Microservices cannot be adopted without continuous integration technology. 100% automated means that the whole deployment and release process is reliable and repeatable. It does not depend on the presence or the good health of certain team members. Everyone should be entrusted to push the code in production. And also everybody should be responsible for the consequences. If things go wrong, you want to have a feedback loop which makes it possible to observe what is happening, detect problems without or with the help of your users so that you are able to react to them.

We saw, for example, that as soon as you activate the new release you have to run a smoke test. You have to be able to quickly check if you did something stupid and you



want to actually pull the plug and revert back to the previous stable state. You want to learn from your users, so you want to be able to do experiments with A/B testing.

In general, choose the right metrics so that you can measure the way your system operates. You want to quantitatively observe and track users. This is no longer a piece of software that someone in some unknown corner of the world takes off the shelf and installs on their machine which is disconnected from the Internet. Users instead are going to interact with you every day. You get to know who they are and you can build a profile and you can remember everything they ever did. With this knowledge you can try to do all kinds of analysis that can give you actionable insight into – let's put in a positive light – how to improve your system so that they become more efficient at what they do. A not-so-positive consequence now that you can observe and track your user is to study their behavior and experiment with ideas on how to get more money out of their pockets, to increase their engagement or addiction with your service, or to nudge them towards paying for those extra features they may not actually need.

We've seen also that the runtime environment has not only to be highly available, but we can expect that it would achieve elastic scalability: given a target performance and by observing the workload, it can allocate or deallocate the resources that are necessary to deliver that level of performance. As the workload is something that you cannot control or predict, then you have to be able to adapt dynamically to changes in traffic.

What is also critical since you are doing these changes very frequently, you do not want anybody to tamper with your build pipeline and inject malicious code into it. You have to be really careful about the dependencies, where they come from and whether they do something more than they claim to do. Today's tools will happily fetch the dependencies, package them together with your software as you ship it and deliver it to be deployed everywhere. You have to trust the quality and safety of this code that you depend on. More and more attacks nowadays are not focused on the systems in production because they are well protected, after all the Cloud is secure, data centers are hard to get into. But maybe the developer machines are weak from a security standpoint. Maybe the build servers and the continuous integration pipelines are weak, so you attack those. And that's when you can inject into the new release, the backdoors that you can use later to get into the system and exfiltrate sensitive information. You should invest into securing not only the production environment, of course you have to protect the data you have to protect the servers, but you also have to protect your build pipeline from which a stream of fresh software releases flows on the servers.

As you can see, most of the concepts that we have introduced this semester are applicable to microservices.

# Feature Toggles

*How to minimize the cost of undoing changes?*

compose features using configuration

Activate or deactivate features with configuration switches, without having to change, rebuild and redeploy the code

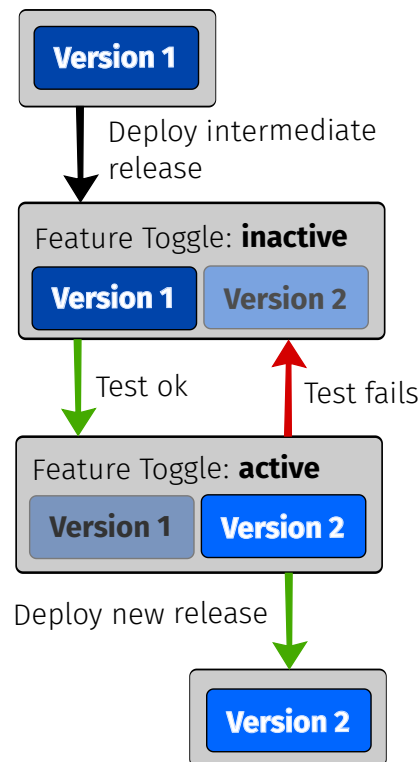
Let's now focus on this particular technique for gradually and gracefully introducing change into an architecture. Feature toggles help to keep the cost of introducing changes low and also help to easily undo the effect of problematic changes.

You're writing some code but you're not so sure that this code is going to be an improvement. You should design your code in a way that it is as cheap as possible to add or remove the feature. After you check if it works or if it doesn't, you can still quickly go back to the previous version.

To do so, configuration comes into play. We use configuration to postpone making design decisions until our users are ready to make them.

Features can be activated or deactivated (toggled) as part of the system's configuration. The configuration composes the features that you want to use in production. Every feature in your code is associated with configuration toggle, with a switch that you can activate or deactivate. By definition, a configuration change is cheap because it does not require you to recompile and reinstall the system, but only in the worst case to restart it.

# Feature Toggles



Given a stable, configurable system we can produce a new version with a new configuration option. Activating or deactivating such option is a very simple way to make the feature come alive or remove it because it doesn't work.

We are in this transition phase, going from a version to another. We know that when we make this evolution step, the change is risky.

We're going to build three different versions of the system. We start from the initial version, where the new feature is missing. Then we have a version in which actually both versions are embedded and the new feature can be activated or deactivated. And then eventually we get to the new version, in which the new feature is stable, always present and running.

Such hybrid, intermediary version is the one that contains also the feature toggle. In the initial phase we are going to introduce the change, release it, but keep the change inactive. In reality, we're still running the old version. Users do not see the effect of your change yet. But the change is already baked into your system. You may ask: what's the point of introducing a change into a system, building it, releasing it, but not using it?

Why would you want to go through all that? It is worth doing it, if you're not yet sure about the new version. You make a so-called Dark Launch. You launch the new version, but the new feature is not visible. Still, you can check if the mere presence of the new code has an effect on the previous version. You can see if there is some conflict between a version assumed to be stable and the new code that you have added. The code is not active, but if the original version is stable and this hybrid version is not stable without running the new code yet, you know that there is already some problem. If you detect some instability even before you activate the feature toggle, you can already go back to version one and try again.

If, on the contrary, the dark launch result is still stable, then what you can do is to finally switch the toggle. Now we go into a different state in which the new version is active, so users can actually try to use it. And then you can observe what happens. This transition is very cheap: just flip one bit to change and activate the new feature. If you have a problem. It's again only one bit that you have to switch to revert back to the original version.

This idea really works if the switch between the two version is fast. If you have to rebuild your system to activate the feature then this is not a feature toggle. Feature toggles are part of the configuration.

If you're happy with the new version, then of course you can really make a stable new release and it's highly recommended to throw away the old version. Why do you want to throw it away and not keep the feature toggle around? It's meant as a rhetorical question: You don't need it anymore because now you trust the new version.

But if you want to keep your system flexible, why not be able to activate and deactivate every feature all the time? What if there is a version 3? Yes, you should be able to switch between version 1, 2 and version 3. Keep in mind that this is not necessarily a version toggle, but it's a feature toggle. So for every feature that you introduce in your system, you should be able to activate or deactivate the feature. How many possible versions are there as a result of all possible combinations of each feature of your system? Here we have two versions because we have one feature that can be active or inactive.

Imagine that you have  $F$  independent features and you want to control each of them independently. How many versions do you need to test? How many possible versions or variants of your architecture are there? Managing  $2$  to the power of  $F$  versions exponentially grows to be a large number and it is simply not feasible to maintain such large and fast-growing configuration space.

Feature toggles is a great technique to have flexibility during transitions between stable releases. You may think: once we built the feature toggle, we might as well keep it over the entire history of our system, you know to keep the architecture flexible. Ok, but you might end up with a combinatorial explosion of possible configurations, possible feature combinations, and if you really have to run tests for all of them, it will be very expensive and take longer and longer to check all possible versions. So expensive that you will probably not do it exhaustively and only manage to test the most frequently used feature combinations.

There's historical evidence of systems that failed because they forgot to remove the feature toggle. After a number of years, somebody by mistake deactivated or activated an old feature toggle and this led to a major failure in production. The toggle was still there, but that particular combination was untested and they learned it wouldn't work the hard way.

# Feature Toggles

- How to implement?
  - Toggle Points: if, #ifdef, late binding, strategy, factory
  - Toggle Routers: Configuration File, Configuration Store, Environment Flag, Command Line Argument
  - Context: A/B testing, Canary Releases
- When to toggle? On build, deploy, startup, live
- How to test? Always run tests on both sides of the toggle
- Feature Toggle Combinatorial Explosion: remove toggles after features become stable to keep the release validation complexity under control and avoid potential unexpected feature interactions

There are many ways that you can use to implement feature toggles. We can use compilers, macros, simple if statements, late binding, virtual methods. Are you familiar with the strategy pattern? You can also use the factory pattern to create an instance of different classes depending on which feature is activated.

In general, you have variation points or branches into your code which you can use to program how to toggle between an activated or deactivated feature. How do you control what gets executed at those points? For example, an if statement is testing a value that comes from a configuration file. This assumes that somewhere there is a file that you specify using some language (nowadays YAML, but doesn't have to be). This configuration file may be centrally managed in a repository or in an operating system registry. Depending on the platform there are conventions for locating config files, environmental variables, command line settings for each particular deployment (e.g., development, testing, staging, production). There can also be specific databases that are just used for storing configuration settings.

Configuration settings can be persistent, but also specified or overridden at startup through environmental variables. They can be passed as command line arguments. Some configuration changes can also be dynamically applied by the end users based on their preferences.

For example, canary users are able to activate or deactivate feature toggles. They signed a special agreement to indicate they are willing to risk it and run experimental features on the latest version. They are aware that some feature toggles could be unstable but they're willing to try it, knowing that they can easily revert back to the stable version by changing a few flags.

This can also work with software as a service: when we get requests from such user, based on their profile configuration we will activate the feature only for them, while all the other users will run in the previous configuration with the feature that is not acti-

vated. This feature toggle is not about changing a config file option, but is about associating the feature toggle with the user account or the API key that is sending you the requests.

Depending on how the feature toggle is built we can constrain who can flip the switch and when this can happen. Build time feature toggles typically use macro expansion or similar metaprogramming constructs. Changes of configuration files can be done at deployment time by installer scripts prompting operators. At startup, the system can read command line arguments or environment flags. We can also ask users live, and this helps to combine feature toggles with A/B testing or canary releases.

How do feature toggles interact with testing and quality assurance? Before and after you activate a feature toggle you should run the tests. Green tests are a pre-requisite to switch it on. Green tests are needed to keep it on.

After you are confident about the new feature, to avoid the feature toggle combinatorial explosion, remove the toggle. Once you trust the feature burn it, hard-wire it into your system so that you cannot go back because you don't need to go back anyway. The more feature toggles you have open at the same time, the more variability you have and the more difficult it is to keep unwanted feature interactions under control.

# How small is a Microservice?

Full control of the devops (code, build, test, release, deploy and operate) cycle by one team

**Iterate fast:** Many small frequent releases better than few large releases

**Rapid Evolution:** If you have to hold a release until some other team is ready you do not have two separate microservices

**Avoid Cascading Failures:** A failed microservice should not bring down the whole system

**Focused Investment:** Scale and Replicate critical parts of the system as opposed to the whole system

Let's go back to the discussion about microservices and their size. What do we mean by small? So how small is a microservice, really?

The limiting factor is how many developers you have or how many teams of developers are there to build your architecture. Each team is responsible not only for writing the code, but is also responsible for building and testing it, for making the release, deploying it and operating the microservice. We are talking about a DevOps cross-functional team.

One of these teams will have between 5 and 10 people. What's the largest possible piece of software that a team of about 10 people can assemble? This already gives you a limit on how large each microservice is going to grow. Then you can ask: is every team going to run their own microservice? Or there are ambitious teams responsible for multiple microservices? For sure, you will not have more than one team working on the same microservice. This would contradict the autonomy principle: every microservices is independent. Therefore you have an independent team that is going to both develop and operate it.

Size also impacts the speed at which the team can iterate and incrementally grow their microservice.

Let's consider a client/server architecture. If you make a change in the server, can you release it before the client has been upgraded? If client and server are developed and operated by autonomous teams, one group can change the server without asking the client team for permission. You have autonomy. You have the independence that you need to set up a microservice architecture. If one microservice team is ready for the release but they have to hold the release to ask for somebody else's permission then they're not really independent microservices. There is a point in which their processes block, their DevOps cycle has to synchronize with the other.

This is a very simple behavior to observe: We announce we want to make a change. We will push the release tomorrow. If somebody calls you in the middle of the night and says please, please don't do it, because otherwise we're going to be severely impacted by the change, then you do not have a micro service architecture.

If you can make a change whenever you want wherever you want, then you manage to keep your microservices decoupled and independent. This property is related with the size, but it's really a different property.

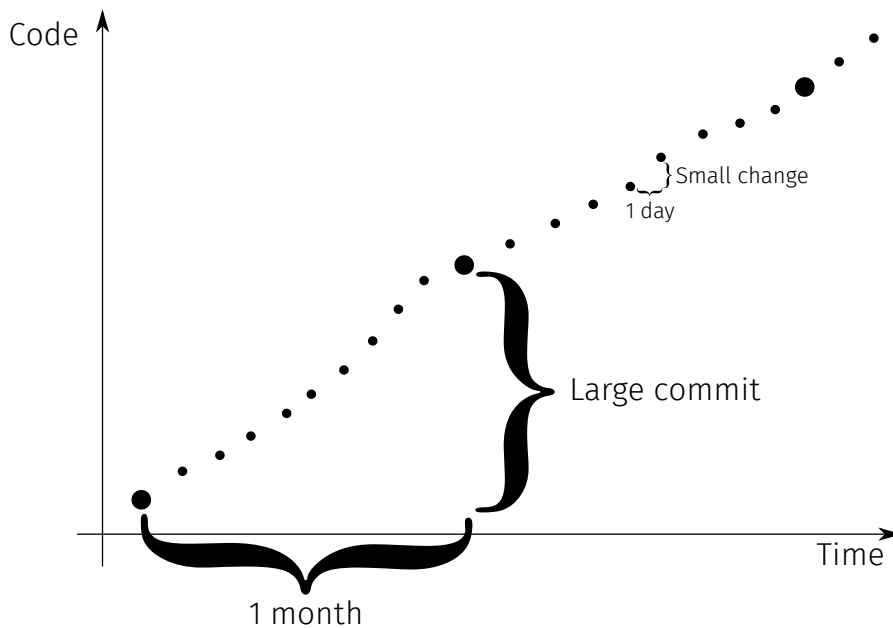
So far we have discussed examples of changes at design-time all the way to deployment time. Let's talk about availability and run-time failures. One of the advantages of microservices is that each microservices can fail independently of the others. If one fails, then the rest will survive. While this is also known as partial failures, the idea is to avoid bringing down everything if just one element fails. We've seen how to achieve this with redundancy and also by introducing, for example, circuit breakers or asynchronous messaging. These type of solutions help the whole system to survive loss of availability of individual elements which can come and go, they can go up and down, but the rest of the system should not be affected. If you have a monolith, the Monolith has the advantage that you don't have partial failures. Either everything works or nothing works. Of course, if nothing works we have a major problem. With microservices, the idea is that most of them would work most of the time. So users can still access the applications and their workflow is not entirely compromised if only a few features or activities get disabled from time to time.

If you have a hot spot – an element of your architecture that becomes a performance bottleneck – you know where to invest to provide the additional capacity. Microservices also help with scalability because can control each independently and decide how many resources to allocate: how much computing power, how much storage, how much bandwidth should be given to run this particular element. Overall this can be more efficient as opposed to having to scale a large system. If you focus the investment, it would be cheaper than having to come up with a fully redundant deployment for the entire system, as opposed to just few critical elements.

These are all different forces that help us to keep the size of the microservice in check because you want to encourage high frequency, incremental releases, therefore keep the changes small. You want to grow the microservice, but not too much, otherwise you would have to deal with all these synchronizations with other microservices that will block your continuous stream of releases. You want to have something small that can fail, but the rest can survive. If you make it too large there will be a limit on how much you can grow. If you grow too big then if this part fails, maybe it's so important that the rest cannot survive and most users will start to notice as it may take a long time to recover. The same is true for scaling: small microservices which become hotspots are cheaper to scale up and scale out.



# Continuous Evolution



Another measurable aspect is the amount of change applied to a system. While the overall system can be large or small, the change that we apply to it should be kept small. Is it more likely that to make a change often, the change should be small. Only by changing less frequently we can afford to apply larger changes.

On this picture we show the size as a function of time: the history of the growth of your system measured with some metric counting the size of the code. It does not necessarily grow monotonically. In most cases, the size will grow as more code, more features, more extensions get integrated. But some changes may refactor, drop some features, consolidate and reduce redundancy by shrinking the system.

Each point is measured when a new version is released. The vertical difference shows the growth in the code, while the horizontal distance indicates how long it took to write, build, test and release the new version, how long it took to apply the change.

We can make a big jump from the initial 1.0 version to the new 2.0 version. How long did it take? Months, years? How much code was added or changed? Thousands, millions of lines of code?

The alternative is to follow a continuous evolution path, in which we see that changes are happening every day. But since we push multiple changes every day, there is only so much that can change in one day. The amount of change will not be as large but it will happen more often.

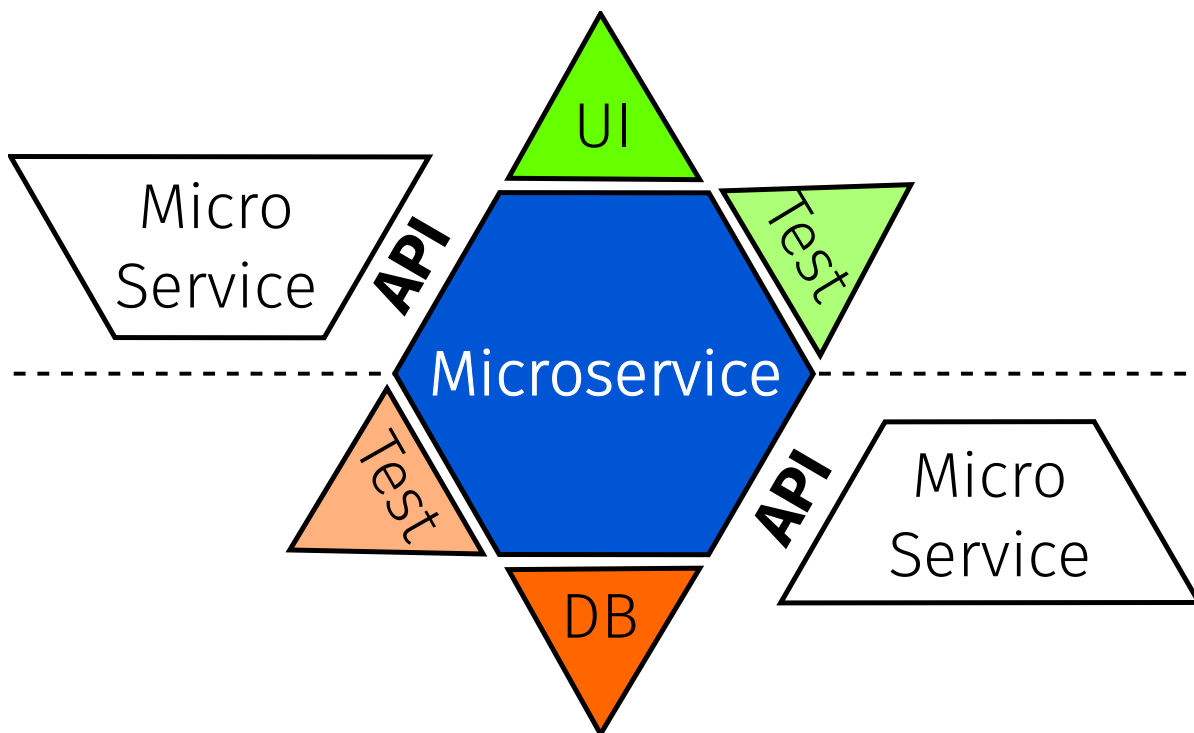
Which strategy will help you to reach the goal faster? A few big jumps or many small ones? Are you sure you will end up in the same spot? In this chart, these two trajectories are somehow overlapping: we may either make a big jump or a lot of small jumps, but we actually end up in the same place. Also, we hope that we land in the right place.

What does a release bring? The opportunity to learn whether the work invested into it works in production with actual users. Every time we make a new release, we have a chance to fail, recover, and learn; whereas if you make one Big Bang release, you make a big jump, the probability that you fail is much higher given the larger amount of change involved. If you have to recover, maybe you have to work for a long time to fix it and

you will still struggle with the quality of a large fix. Also, experimenting with so many changes together in one release will make it more difficult to observe the relationship between individual changes and the resulting user satisfaction.

We have illustrated that small size of changes helps to gain confidence at every step and iterations can thus proceed at a higher rate. The increased overhead of making more releases (with small changes) compensates for the increased risk when making fewer releases (with large changes).

# Hexagon Model



Microservices are often visualized as hexagons. Where does this hexagon visualization come from?

When representing an architecture either in the logical view or also sometimes in the deployment perspective, you will see this shape associated with microservices. It looks great, since you can tile many microservices side by side and it will look less boring than the usual boxes.

However, the hexagon as a metaphor for components is even older. The idea is that with a hexagon we have six sides and different sides represent different contexts in which the component can be used.

The main vertical axis represents the production side of your system. When you put a component in production it means that you have a user interface on top. If the component is stateful (most of them usually are) it will use a database shown at the bottom to store persistently its state. So this is the production axis: users access and process their actual data.

Before you get to production, you are staging and testing your component and to do manual tests through the user interface is expensive. Instead you run automated tests that control the component. These are often encapsulated in test drivers and are shown attached to the top right side of the hexagon. They exercise the code trying to cover 100% of all the paths. When you run the tests, you shouldn't do it on top of your production database and potentially corrupt all the data. Instead, you want to attach as shown on the bottom left side a copy of the database dedicated for testing. This has the advantage that you can write test oracles which have expectations about the initial state of the component. The visualization shows that for testing stateful components, they will be attached to a test database and then the tests drivers will use it to check that expectations are fulfilled when the component is using the test data. This is the quality assurance axis.

But this is a hexagon, so we have one more axis. That's the integration axis. That's

where we find the API of the component which will be used to integrate it with other components.

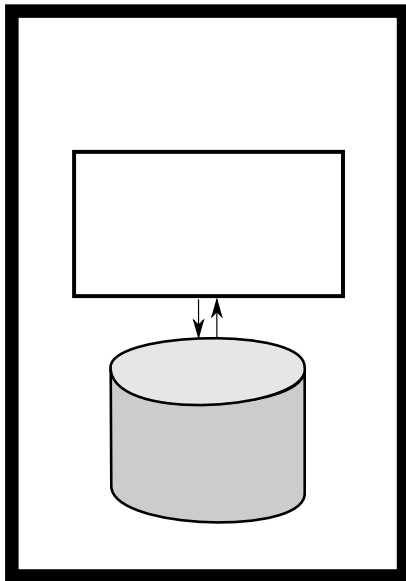
The top left side represents the interface provided by our microservice to the rest of the microservices which require it, while the bottom right side shows the opposite dependencies, the ones required by our microservices and satisfied elsewhere in the architecture.

For unit testing, the microservice wouldn't work without its dependencies: in the same way stateful components will be bound to a test dataset, we can connect the microservices to mock microservices, which will then be switched to the real dependencies for integration testing.

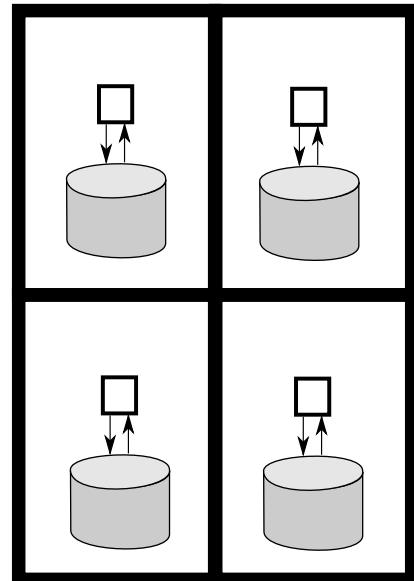
This hexagonal component model – recently adopted for microservices – helps us to distinguish three different types of interfaces along the production axis, the quality assurance axis and the integration axis. In general, elements attached to the top sides will depend on our component, while elements attached to the bottom sides represent our own dependencies.

# Decomposition

**Monolith**



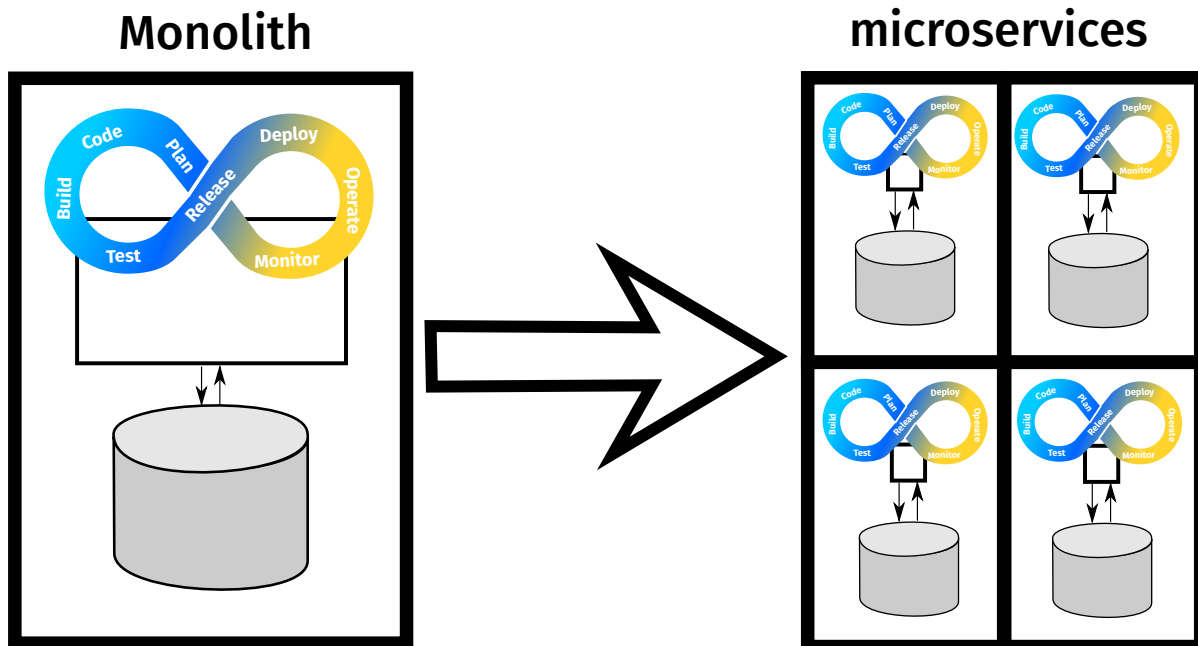
**microservices**



When we introduce microservices, the big challenge is how we transform an architecture with a single element into one with multiple ones. While this is relatively easy to do for the code following the good old maximize cohesion and minimize coupling rule, it is more difficult to split the database. It's more difficult to split a large schema with many relationships into many small ones.

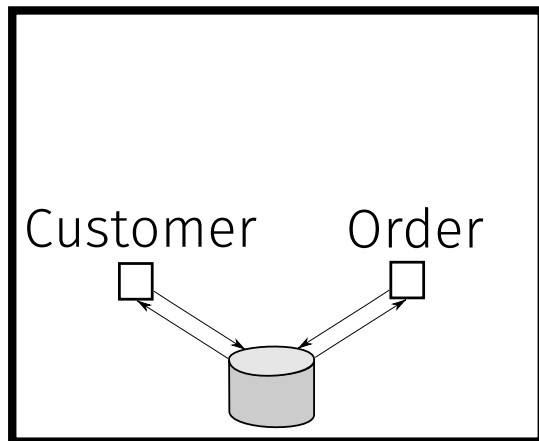
Once you have decomposed the monolith into multiple microservices you also need to recompose them together using the right type of connectors. But first we go in the opposite direction. We start from something large and we try to break it into little pieces.

# Independent DevOps Lifecycle

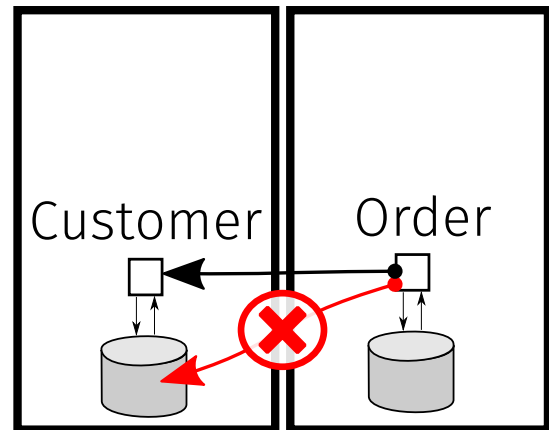


Why do we want to do that? Because this way every one of those pieces will follow its own independent development and operation life cycle. You will be able to introduce changes into each of the microservices without affecting the life cycle of the others. Instead if you want to make a change in the monolith, you have to stop everything and you have to restart the whole thing every time, and every change can potentially affect everything else in the architecture, since there is no structure to contain and absorb the impact of changes.

# Isolated Microservices



**Monolith**



**Microservices**

One important constraint that is also part of the microservice definition regards what kind of connections we can establish between. In this picture we can see that we are using a shared database connector between two different components. They use the same database to store their state as well as they can also communicate through this database. The boundary surrounds both of them: on the left we are still talking about a modular monolith. It is deployed and it uses a single database, but the code inside is already modular. We can already distinguish the different subcomponents.

If we break this into microservices and then each of the code components will have their own independent storage. The question is: how do these elements communicate? There are two possibilities.

The Black one represents the correct solution in which one microservices asking the other one for some information. The red arrow instead is bypassing the API of the microservices and directly going to the database to fetch the data. This mimics what originally happened in the original version. Here it was allowed because the database was shared. With microservices the data is not meant to be shared at all, so using this connector is actually a mistake.

Why is this such a big deal? It has to do with flexibility - today's topic. It has to do with how easy it is to introduce changes into the internal representation of the data owned by each microservice. In both scenarios we have a schema for the database that will be shared between the two components if both components access the same database. With microservices we want to avoid that because if we do not share the schema, we can change it without affecting external components. However, if a component directly reads the information from the other database and we change the schema of this database, the component will be affected.

Microservices should encapsulate their database behind their API so that from the outside you can still access the information, but you do not make any assumptions about the way that they are represented inside. The constraint recommends using appropriate layers for having a compatible representation of shared data from the outside and keeping the freedom to change the way it is represented inside the microservice.

# Isolated Microservices

For us service orientation means encapsulating the data with the business logic that operates on the data, with the only access through a published service interface. No direct database access is allowed from outside the service, and there's **no data sharing among the services**.

Introducing microservices requires to keep them isolated. This means that you use encapsulation not only for the business logic but also for the data. The only way that you can access the data inside the microservices is through its API, the service interface. There is no data sharing, no direct database access is allowed.

This is one of those rules that if you want to call your architecture a microservice architecture, you have to follow. If for some reason you bypass the interface and you directly extract the information from the database, you're violating this design constraint. In the short term, maybe you get an advantage. Maybe it's faster to just send a query to the database to read the data that another microservice needs. In the long term this will create a problem, because it will prevent these microservices from independently evolving the internal representation of its data.

The boundary you draw around the microservice does not only include the microservice and its data but it should include all other microservices which share some assumption about the data, which is no longer private as it is hidden inside the microservice but has become shared potentially among all microservices of your architecture.



# Splitting the Monolith

1. Start from a working system with a modular, monolithic architecture
2. Decompose it into microservices as it grows too large for a single team

It is faster to start with a working centralized system

Later it becomes easier to see where to draw the boundaries

Martin Fowler

How can we design a microservice architecture? Should we start from scratch introducing microservices? Or should we split an existing Monolith?

Let's start from something that works but is just a single element. We notice that it is now growing beyond the capacity of a single team. We hit a limit and we can no longer manage the growth of this particular element. It is just too large and our velocity is slowing down. This is a good starting point for decomposing it.

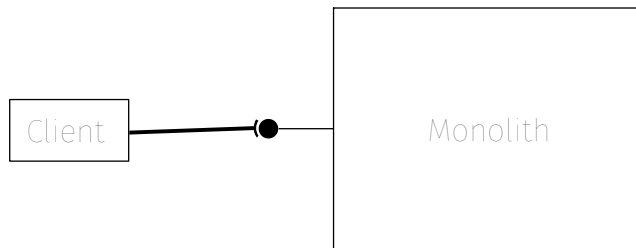
The message is: don't start from scratch with microservices, start with whatever it takes to get it working. And once it works then you can improve the way you can evolve it in the future by introducing the practices we have discussed so far. Microservices are better for evolving our system, but they're not as fast to start a new system from scratch.

How do we make the transition? Once you have a system that works but that is centralized, then you can start to see which possible boundaries you can draw around its internal components. Along which lines can we split it?

This is an example of an architectural refactoring of a system. We are not going to refactor the code. We will refactor the design so that we can make a transition from a monolith that has grown too large into multiple microservices. As we split it into different parts, the constraint is that we want to keep the client working during this transition. From the client perspective, the system should be available all the time.

It's like when they do some construction work on the highway but they cannot close the highway. You want to keep the cars driving through. Or maybe a more appropriate image regarding software is: you are flying an airplane and you decide that you need to make a few changes but you cannot land anywhere to replace the engine.

# Splitting the Monolith

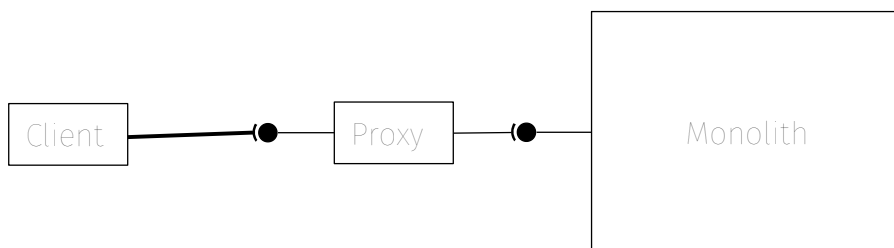


Assumption: the Monolith should have some kind of interface

At the beginning we have a monolith. And we have some kind of an interface that allows the client to interact with it.

This is the initial state of our architecture. The first thing that you should do is to isolate the client from the system where you're working on.

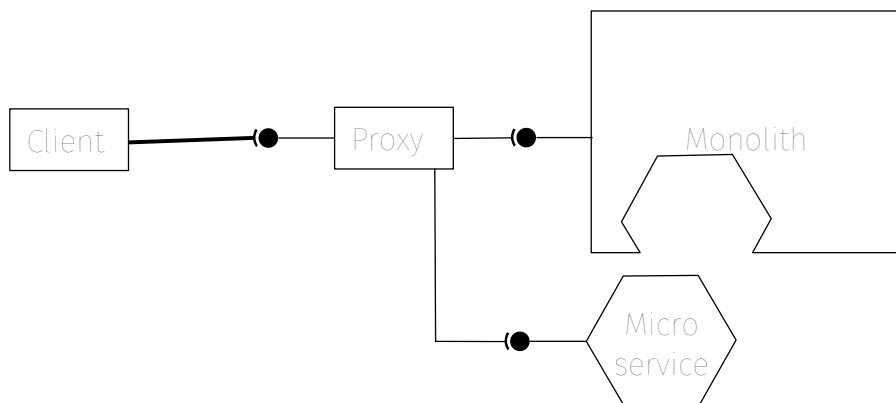
# Splitting the Monolith



Add an intermediate layer to intercept and redirect traffic

You should then introduce an intermediate layer so that we can control the communication between the two parties. We can intercept it and redirect it. This is done usually through some kind of proxy. The HTTP protocol supports proxying for free. If the protocol is more complex, then you have to find a solution. In the worst case you can resort to a man-in-the-middle attack.

# Splitting the Monolith



Carve out the first microservice and redirect requests towards it  
To test if replacement works, run them in parallel and compare results

Assumption: the microservice is stateless

Now that you have intercepted the traffic, you can start to carve out, extract, and package the first microservice. Redirect the relevant requests towards it.

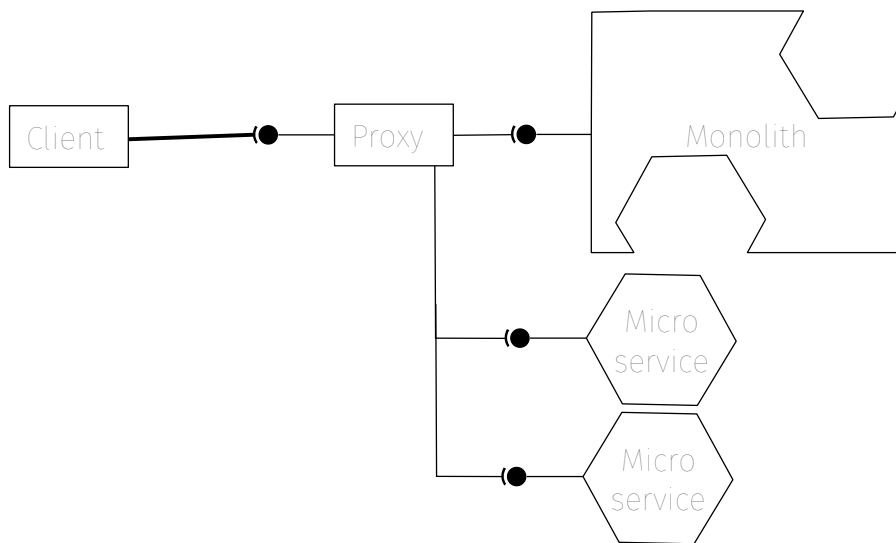
The client depends on certain interface. Part of this interface is a subset we can implement in the microservice. When the client wants to access that part, we will send the request over to the microservice; when the client wants to access everything else, it still goes back to the original system, from which here we see that we have been cutting off a small piece. In reality the piece is still there, but we don't use it anymore. Typically the monolith you actually never touch, you are afraid to actually change it.

If you want to test how successful the replacement was, you can take the request, send it to both sides and then compare the results. If the microservice and the monolith agree, then you know that you have made a good replacement. If the microservice sends you back a different result, you can blame it. The monolith is the ground truth. This helps to catch mistakes in the transition using the original implementation as the oracle.

This is very easy to do if the microservice is stateless. You can just take the code and put it in a new container; there is no state that you have to migrate. For stateful microservices you also need to partition the database. You will need to extract a subset of the schema which will be encapsulated within the microservice.

If you need to migrate part of the state of the monolith and bring it into the microservice you can still follow this process, but at some point you will need to decide which fork of the state will become the master version. While at the beginning the state within the microservice is likely to drift and become incorrect. It will need to be often reseeded based on the master copy maintained by the monolith. Eventually however you will stop updating the state in a monolith after you fully trust the correctness of the microservice.

# Splitting the Monolith



Keep extracting microservices (most valuable, least risky first)

After first step, the first microservice is working. The client didn't notice the switch. We can try to repeat the process as many times as needed.

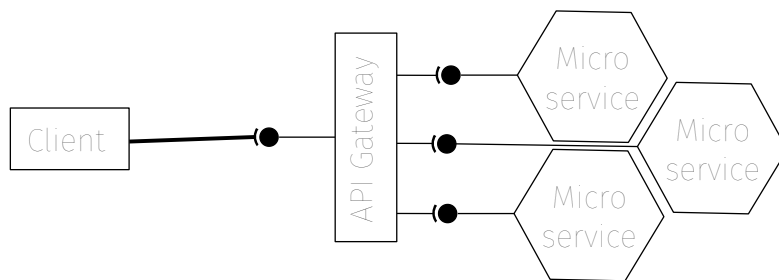
Here's the second piece. We have prioritized the work to extract the most valuable piece of code and the one that is least risky to repackage as a standalone microservice. Sometimes during the transition we're actually rewriting the code like from COBOL to Java.

# Splitting the Monolith

This migration can be done gradually without affecting clients

Much easier to split stateless components than stateful ones

Only at the very end the monolith can be retired



This major undertaking can take a long time, but eventually you have managed to trick the client into believing that is still talking to the old Monolith, while in reality behind the scenes you have your microservice architecture and you have retired the Monolith. This process can be done slowly and gradually and every step along the way you can check that the clients do not notice.

# Microservice Best Practices



In this lecture we have seen how to make architectures flexible. This included the microservice architectural style. The term emphasizes the small size of the software components, with some even bringing mini-services or nano-services into play.

In my humble opinion, size doesn't really matter. What matters is whether and to which extent the elements in your architecture are coupled from each other. If they are loosely coupled or not coupled, they can change independently and they can change often. Does size impact coupling? For sure, but you can also have large components which have very low coupling and therefore can evolve independently or end up with many small and tightly coupled components.

Another heuristic related to the size is known as the "2-pizza team". It should be possible to feed each microservice team with 2 pizzas. In Italy this would translate to maximum one developer and one operator. Elsewhere pizzas can feed a few more people.

As we have almost finished this lecture, let's go back over the history of software architecture and see where some of these ideas that are being very popular nowadays with microservices come from. We will see that they are actually much older than the buzzword they are associated with. This is a very common trend in our industry: repackaging old ideas with new labels. It is easier to keep selling new technologies if you are just changing their names. For students, this might be hard to believe because you're trying to learn the current state of the art, but after a few years of industry experience you will start to observe these paradigm shifts sweeping through the technology landscape. You should realize that the problems you are trying to solve always remain the same, so you can just learn how to match problems with solutions no matter what they are called today.

# Bezos's Mandate (2002)

1. All teams will henceforth expose their data and functionality through service interfaces.
2. Teams must communicate with each other through these interfaces.
3. There will be no other form of interprocess communication allowed: no direct linking, no direct reads of another team's data store, no shared-memory model, no back-doors whatsoever. The only communication allowed is via service interface calls over the network
4. It doesn't matter what technology they use. HTTP, Corba, Pubsub, custom protocols – doesn't matter.
5. All service interfaces, without exception, must be designed from the ground up to be externalizable. That is to say, the team must plan and design to be able to expose the interface to developers in the outside world. No exceptions.

Back almost 20 years ago, even before when the cloud was starting to appear, there was a company in which one day, all the IT department got to work and received an email from their boss.

The email informed them that from now on, every software developed in the team will have to be exposed through a service interface. If you want to call other software provided by other teams, you have to go through this interface. If they want to call your software, they have to use its service interface. There is no other integration mechanism allowed. You cannot directly read the data from somebody else's database. This is forbidden. You cannot share data through memory. You cannot come up with any other kind of back door. It is only possible and allowed to communicate through the front door: a service interface. This is given as a constraint on an abstract level: call the service interface across the network. It's ok to use HTTP, CORBA, MQ, gRPC, you choose the protocol, you pick the most appropriate connector. There is some flexibility in the choice of connector, but you have to go through the service interface.

This constraint created a billion dollar business out of a very simple design decision. All service interfaces without exception must be designed to be externalisable. What does it mean? We need to be ready to support both internal and external clients. It means that once you have an interface. Everyone may call the service. You can first build something for internal use, and then later you can open it up to the rest of the world so that, for example, you can make money with it. No exceptions.

If you, by the way, ignore these rules, you will be fired. That's not part of the slide, but it was actually part of the email.

This impulse started the service-oriented architecture trend which eventually evolved into cloud computing, everything as a service and microservices.



# Evans's Bounded Context (2004)

There is no single uniform model to design large systems. Models are only valid and consistent within a given context boundary. Some translation may be needed to cross boundaries.

From a similar time frame, domain-driven design (DDD) is also highly relevant today, as with microservices we need to find a way to decompose a large system into small pieces.

There is a limit not only in the number of people that you can work with to support the system, but there is a limit also in the conceptual integrity of the content managed by the system. If its domain grows beyond a certain size, its complexity will also grow as you try to cover within the same system concepts that are becoming increasingly different.

Consider your natural language understanding skills. For example, you are born in a certain village, you go to primary school everybody speaks the same language. Then one day you will take a walk and you go to the next village and you start to realize that people speak in a slightly different way. Eventually you grow up and move to the city. Your accent is different than people can tell where you come from. One day you cross the boundary to a new country and there the language is completely different.

The same happens with software systems: when they are young and small, they're very consistent because they just focus on doing precisely something that is very narrow. Eventually they grow by adding more features, more concepts, more data and eventually they hit a boundary. They need to inter-operate with another system and that's when you start to need to have a translation because the other side speaks a different language to represent shared concepts. You can hardly mix data on the inside with data from the outside.

Same holds with systems that are initially used only by one user. As the user community grows, there is more and more pressure to support different use cases, personalized scenarios. With a sufficiently large user population, it is impossible to find the only way to do something or to keep doing things in the same way for everyone. The only way to keep growing is to manage diversity by drawing boundaries.

Is there ever going to be the only Universal solution? Or just many failed attempts at over-generalization.



# Bob Martin's Single Responsibility Principle (2003)

A class should have only one reason to change.

Gather together things that change for the same reason, and separate those things that change for different reasons.

This is also a similar concept for keeping your components cohesive: the idea of the single responsibility principle states that if you have a piece of software – like in this case a class in object oriented programming, but one can also extend it to a component, or a microservice – it should only have one reason to change.

If things change for the same reason, you keep them together (high coupling). And if things change for different reasons, you separate them (low coupling)

This abstract advice can help you with your monolith decomposition efforts. Think about who would request possible changes, trace their impact across the architecture and bring together all elements that can be directly or indirectly affected by the requirements provided by each key stakeholder.

# UNIX Philosophy (1978)

Write programs that do one thing and do it well.

Write programs to work together.

Write programs to handle text streams, because that is a universal interface.

Design and build software to be tried early, ideally within weeks.

Don't hesitate to throw away the clumsy parts and rebuild them.

M. Douglas McIlroy

We keep going back in time even further. In the early days of UNIX, they started to reflect about its design. They didn't call it architecture, but they wrote about the design philosophy of the system. These ideas also resonate with today's microservices, back then they were called programs.

When you write some programs, each program should do one thing. And do it well. There is no need to replace them with some other program if you have the best program to do a certain thing, to solve your problem.

However, since they only do one thing, sometimes you have to combine them to actually do something useful. Since you cannot anticipate and should not constrain how components can be composed, you have to write them so they can offer a universal interface. Back then, the universal interface was textual data streams: standard input and standard output. We still have it today. For composing distributed components, HTTP could be considered as a universal interface.

As you start with a new project: design and build software to be tried and tested as early as possible, ideally within one or two weeks. Do not wait for end of the project to deliver the first result that finally works. You should have as early as possible, and then you can keep growing it. Today we call this a minimum viable product (MVP).

Don't hesitate to throw away the clumsy parts and rebuild them. This is also an idea often cited with microservices. Since they are small, you can afford to throw them away and rebuild them. If they become too large, you have invested so much into building them and you will think twice before replacing them. If you hesitate before replacing a microservice, it's already grown beyond the limit of what keeps a microservice small.

# Separation of Concerns (1974)

One is willing to study in depth an aspect of one's subject matter **in isolation, for the sake of its own consistency**, all the time knowing that one is occupying oneself with only one of the aspects.

But nothing is gained -- on the contrary! -- by tackling these various aspects simultaneously.

Edsger W. Dijkstra

The term “separation of concerns” is really old, but still helpful today as we struggle with microservice decomposition.

Once you try to understand how to solve a problem, you want to separate different aspects so that you can solve them in isolation. And this helps you to focus your problem solving skills on something that is manageable. If you try to solve everything together at the same time, then it will be very difficult to make it work.

Also, if you found multiple problems that can be solved independently, you can involve multiple teams of developers to build the corresponding solutions, also known as microservices.

# Parnas's Criteria (1971)

One begins with a list of difficult design decisions or design decisions which are likely to change. Each module is then designed to hide such a decision from the others.

It should be possible to make drastic changes to one module without a need to change others

Only three years after the Garmisch conference where the term software engineering was born, David Parnas proposed his criteria to decompose systems into modules. At that time they didn't call them components. Microservices were yet to come.

You start by making a list of difficult design decisions, or what we have called architectural decisions. Hard to change later on, but which should make it possible to design a flexible architecture.

Each module should be designed to hide such a decision from the others. This will make it possible to change what's inside one module without affecting the rest.

Encapsulation, modularity and information hiding lead to the constraint to use exclusively the service interface as opposed to offer direct access to the underlying implementation. We can rewrite an entire microservice using a different programming language, the rest will not notice because the service interface remains the same.

# Conway's Law (1968)

Any organization that designs a system will inevitably produce a design whose structure is a copy of the organization's communication structure

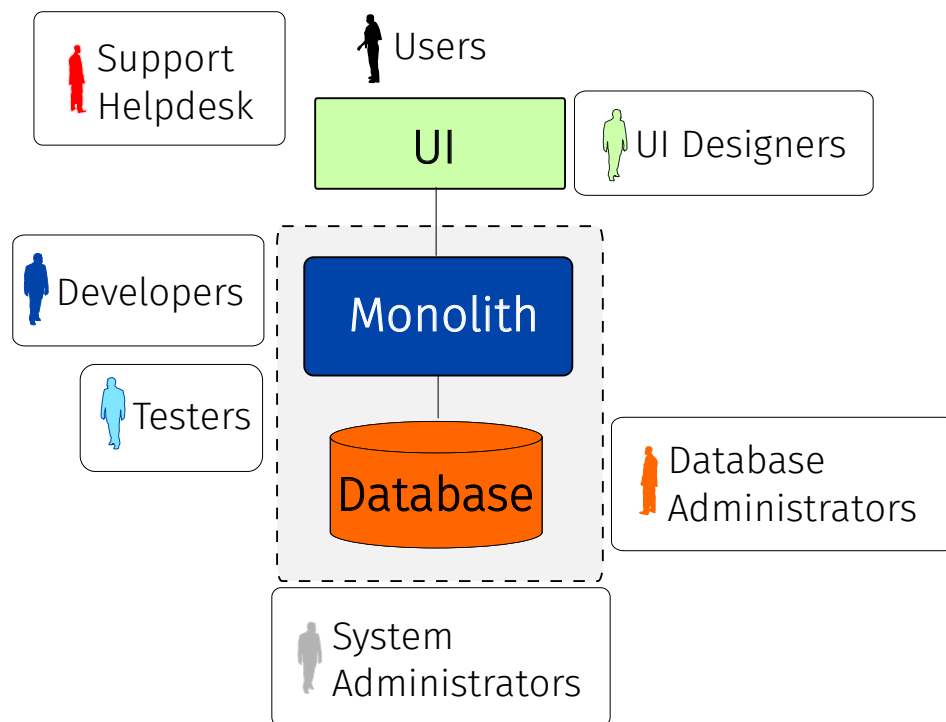
We reach now probably the most cited concept when you hear a microservices talk in the industry: Conway's law, sometimes also mentioned as the "reverse Conway manoeuvre".

Any organization which designs a software architecture will inevitably produce a design whose structure is the copy of the communication structure within the organization itself.

If people are used to communicate in a hierarchical reporting structure, like where every team talks to the manager, the managers talk to their manager, all the way up to the CEO, the software that they design will actually look like a hierarchical decomposition of components.

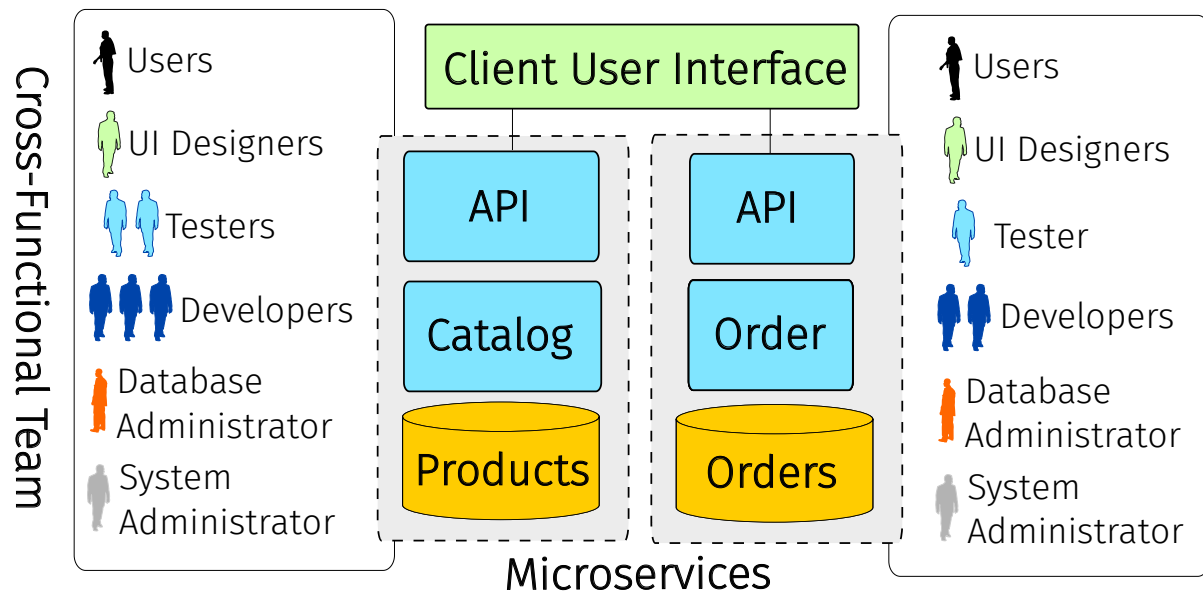
Taken to the extreme, depending on how large the organization is, the number of components in the software architecture will be proportional to the number of developers, the number of offices or the number of floors. If you would like the software architecture to change, you will need to re-arrange the chairs around the table, or the cubicle floorplan.

# Conway's Law (1968)



The monolithic architecture shows an example in which development is kept separate from operation. Developers build the system, they release it, and then they push it over the wall to somebody else who is in charge of running it.

# Conway's Law (1968)



Only by changing the organization of your team, you can actually achieve a flexible microservice architecture. Each microservice is developed and operated by a cross-functional team with enough members to play all of the roles covering the whole life cycle of the software.

# Vogels's Lesson (2006)

The services model has been a key enabler in creating teams that can innovate quickly with a strong customer focus.

Giving developers operational responsibilities has greatly enhanced the quality of the services, both from a customer and a technology point of view.

Why is it important to have cross-functional teams dedicated to each microservice?

Let's go back to where we started from: service-orientation has been a key enabler in creating teams that can innovate quickly with a strong customer focus.

Empowering developers with operational responsibilities also enhances the quality of the services both from a customer as well as a technology point of view.

This is a polite way to say that if a developer gets a call in the middle of the night and they have to fix their service which just went down, the developer will tend to be much more careful in introducing bugs, performance or availability issues in their system as opposed to when it's somebody else's problem to fix.

The same should hold for architects, I wonder why they never get to enter their beautiful buildings and try to live inside them for a day or two.



# References

- Stephan Murer, Bruno Bonati, Frank J. Furrer, *Managed Evolution: A Strategy for Very Large Information Systems*, Springer, 2011, ISBN 978-3-642-01632-5
- Werner Vogels, [Interviews Web Services: Learning from the Amazon technology platform](#), ACM Queue, 4(4), June 30, 2006
- Melvin E. Conway, [How do Committees Invent?](#), Datamation, 14 (5): 28–31, April 1968
- Edsger W. Dijkstra, [On the role of scientific thought](#), In: *Selected Writings on Computing: A Personal Perspective*, Springer, 1982. ISBN 0–387–90652–5.
- David L. Parnas, [On the criteria to be used in decomposing systems into modules](#), Communications of the ACM, 15(12): 1053-1058, December 1972
- Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004, ISBN 978-032-112521-7
- James Lewis, Martin Fowler, [Microservices](#), 2014
- Sam Newman, *Building Microservices*, O'Reilly, February 2015, ISBN 978-1491950357
- <https://www.microservice-api-patterns.org/> (API Evolution Patterns)